



Tribhuvan University

Faculty of Humanities and Social Sciences

A PROJECT REPORT ON

Smart Agro

Submitted to

Department of Computer Application

Nepal Kasthamandap College, Kalanki

In partial fulfillment of the requirements for the Bachelors in Computer Application

Submitted by:

Hitesh Dangol

TU Registration: 6-2-144-09-2021

September, 2026

Under the Supervision of:

Er. Sujan Poudel



Tribhuvan University
Faculty of Humanities and Social Sciences
Nepal Kasthamandap College

Supervisor's Recommendation

I hereby recommend that this project prepared under my supervision by Hitesh Dangol entitled “**Smart Agro**” in partial fulfillment of the requirements for the degree of Bachelor of Computer Application is recommended for the final evaluation.

Er. Sujan Poudel
Project Supervisor
BCA Department
Nepal Kasthamandap College



Tribhuvan University
Faculty of Humanities and Social
Sciences
Nepal Kasthamandap College

Letter of Approval

This is to certify that this project prepared by Hitesh Dangol entitled “**Smart Agro**” in partial fulfillment of the requirements for the degree of Bachelor in Computer Application has been evaluated. In our opinion it is satisfactory in the scope and quality as a project for the required degree.

Er. Sujan Poudel Project Supervisor BCA Department Nepal Kasthamandap College	Er. Sujan Poudel Program Co-ordinator BCA Department Nepal Kasthamandap College
Roshan Poudel Internal Examiner Nepal Kasthamandap College	Yuba Raj Devkota External Examiner Kathmandu Sikshya Campus

Abstract

Agriculture is a major source of livelihood in Nepal, but farming decisions often rely on traditional knowledge and personal experience. Climate variability, unpredictable rainfall, soil degradation, pests, and plant diseases create significant challenges for farmers. This project presents Smart Agro, an integrated web-based smart farming system that combines machine learning advisory services with a digital agricultural marketplace. The system supports three user roles: farmers, sellers, and administrators. It uses Gaussian Naive Bayes for pest risk prediction, Random Forest for crop recommendation, and EfficientNetB4 for plant disease recognition. The system also provides crop recommendations, disease information, medicine suggestions, and access to agricultural products. Built using React, Node.js, Express, and MongoDB, the platform includes role-based access control, shopping cart functionality, order management, cash-on-delivery, and eSewa payment integration. Testing confirmed the proper functioning of the system and its integrated workflows.

Keywords: Smart Farming, Machine Learning, Crop Recommendation, Pest Risk Prediction, Plant Disease Recognition, e-Commerce, eSewa, Precision Agriculture.

Acknowledgement

I would like to express my sincere gratitude to my supervisor, **Er. Sujan Poudel** Sir, for his invaluable guidance, support, and encouragement throughout the development of this project, **Smart Agro**.

I am also thankful to **Nepal Kasthamandap College** for providing the necessary resources, infrastructure, and a conducive environment for completing this project. I extend my appreciation to the faculty members of the **BCA Department** for their valuable suggestions and academic guidance.

Finally, I would like to thank everyone who, directly or indirectly, contributed to the successful completion of this project. This experience has been highly educational, and I hope Smart Agro serves as an effective and innovative platform for sustainable, community driven trading.

Warm regards,

Hitesh Dangol

Table of Contents

Supervisor’s Recommendation	i
Letter of Approval.....	ii
Abstract.....	iii
Acknowledgement	iv
List of Figures.....	vii
List of Tables.....	ix
List of Listings	x
List of Abbreviation	xi
Chapter 1. Introduction.....	1
1.1 Introduction	1
1.2 Problem Statement	1
1.3 Objectives.....	2
1.4 Scope	2
1.5 Limitation	2
1.6 Methodology	3
1.7 Report Organization	4
Chapter-2. Background Study and Literature Review	5
2.1 Background Study.....	5
2.2 Literature Review.....	6
Chapter-3. System Analysis and Design.....	8
3.1 System Analysis	8
3.1.1 Requirement Analysis	8
3.1.2 Feasibility Study	9
3.1.3 Project Schedule.....	11
3.1.4 Object Modeling using object and class Diagram	12
3.1.5 Dynamic Modeling using Sequence Diagram	14

3.1.6 Process Modeling using Activity Diagram	17
3.2 System Design	20
3.2.1 Refinement of Class Diagram.....	20
3.2.2 Component Diagram.....	21
3.2.3 Deployment Diagram.....	22
3.2.4 Wireframes	24
3.2.5 Interface Design (UI Interface/Interface Structure Diagrams)	27
3.3 Algorithm	29
Chapter-4. Implementaion and Testing	33
4.1 Tools Used.....	33
4.2 Implement Details of the modules	34
4.3 Testing.....	37
4.3.1 Unit Testing.....	37
4.3.2 System Testing	38
4.4 Result Analysis.....	38
Chapter-5. Conclusion and Expected Outcome	41
5.1 Conclusion	41
5.2 Outcome.....	41
5.3 Future Recommendation.....	42
References	
Appendices	

List of Figures

Figure 1.1 Agile Method	3
Figure 3.1 Use Case Diagram	9
Figure 3.2 Class Diagram of Smart Agro.....	13
Figure 3.3 Object Diagram of Smart Agro.....	14
Figure 3.4 Sequence Diagram for User Flows.....	15
Figure 3.5 Sequence Diagram for Admin Management	16
Figure 3.6 Activity Diagram for Order and Payment	18
Figure 3.7 Activity Diagram for Admin Management.....	19
Figure 3.8 Refined Class Diagram (MVC Layers)	20
Figure 3.9 Component Diagram - User Side	21
Figure 3.10 Component Diagram - Admin Side	22
Figure 3.11 Deployment Diagram (Local Deployment)	23
Figure 3.12 Wireframe Diagram of Login Page	24
Figure 3.13 Wireframe Diagram of Shop Page.....	24
Figure 3.14 Wireframe Diagram of User Dashboard Page	25
Figure 3.15 Wireframe Diagram of Admin Dashboard Page.....	25
Figure 3.16 Wireframe Diagram of Crop Recommendation Page.....	26
Figure 3.17 Wireframe Diagram of Disease Recognition Page.....	26
Figure 3.18 Interface Design of Login Page	27
Figure 3.19 Interface Design of Admin Dashboard Page	27
Figure 3.20 Interface Design of User Dashboard Page.....	28
Figure 3.21 Interface Design of Shop Page	28

Figure 3.22 Interface Design of Crop Recommendation Page	29
Figure 3.23 Interface Design of Plant Disease Recognition Page	29
Figure 3.24 Working Mechanism of Random Forest Algorithm	30
Figure 3.25 Working Mechanism of Gaussian Naive Bayes Algorithm.....	31
Figure 3.26 Working Mechanism of CNN Algorith	32

List of Tables

Table 3.1 Project Schedule.....	11
Table 4.1 Unit Test Cases and Results	37
Table 4.2 System Test Cases and Results	38

List of Listings

Listing 4.1 Code Snippet of User Registraton	34
Listing 4.2 Code Snippet of User Login	35
Listing 4.3 Code Snippet of Plant Disease Recognition	35
Listing 4.4 Code Snippet of Crop Recommendation	36
Listing 4.5 Code Snippet of Pest Risk Prediction.....	36

List of Abbreviation

AI	Artificial Intelligence
AWT	Abstract Window Toolkit
COD	Cash on Delivery
CNN	Convolutional Neural Network
DB	Database
IoT	Internet of Things
HTTP	HyperText Transfer Protocol
JS	JavaScript
JWT	JSON Web Token
MERN	MongoDB, Express, React, and Node.js
ML	Machine Learning
MVC	Model-View-Controller
pH	Potential of Hydrogen
UML	Unified Modeling Language

Chapter 1. Introduction

1.1 Introduction

Agriculture is crucial to world food security and economic stability, especially in developing nations. However, the industry has continued to experience unpredictable weather, pest infestations, soil erosion and obsolete farming techniques. The problem is augmented by the fact that farmers have poor access to sound agricultural information with many of them still being dependent on their traditional knowledge and intuition which is not usually sufficient in the ever-changing climatic conditions. New technological developments present effective solutions. Machine learning, real-time analytics, and use of modern web technologies can empower farmers with data-driven approaches capable of making better decisions. This idea serves to present the Smart Agro Assistant, a Web-based application that offers tailored farming advice based on artificial intelligence. The app is scalable and open-source, developed on the MERN stack (MongoDB, Express.js, React.js, and Node.js). It uses a Random Forest algorithm to understand soil type, pH, temperature, and rainfall to provide specific crop recommendations. It also applies Guassian Naive Bayes Classifier algorithm to detect the presence of pests, which allows detecting the risk of threats and preventing them as early as possible. The Smart Agro Assistant helps close the divide between the agricultural innovation of the modern and the smallholder farmers. It is also a friendly, web-accessible, and easy-touse tool that can be used on any internet-enabled device, even in underserved regions, unlike many of the expensive advisory systems available.

1.2 Problem Statement

Farmers often face significant challenges that hinder agricultural productivity and sustainability. The first problem is related to the inability to choose the most appropriate crops under special soil and climate conditions that directly influence the yield and effectiveness of resources. Also, there is no immediate, weather-based agricultural guidance, which means farmers are also not equipped to react quickly to the vagaries posed by the weather, especially in regard to making pivotal decisions like irrigation and harvesting. Another prevalent issue is the late identification of the pests, which can be especially devastating to the crops and economies. To overcome these challenges, our solution will be an intelligent, data-driven farming assistant that will be used to offer

datadriven crop suggestions, timely weather-related advice, and pest notices, thus enabling farmers to make data-driven decisions to achieve better overall agricultural performance.

1.3 Objectives

- To develop a crop recommendation system
- To develop an pest risk detection system to accurately identify common pests specific to a farmer's crop type and geographic region
- To develop a plant disease detection system with its cure

1.4 Scope

- Role-based access control for farmers and administrators.
- Crop recommendation based on soil nutrients (Nitrogen, Phosphorus, Potassium), temperature, humidity, pH, and rainfall using a Random Forest machine learning model.
- Plant disease recognition using a CNN, with disease causes and treatment recommendations.
- Pest risk prediction using a **Gaussian Naïve Bayes** model based on environmental and crop-related parameters.
- An online marketplace where farmers can buy and sell agricultural products.
- Product management, order management, and user profile management.
- Digital record management for farming activities and recommendations.
- A responsive web interface that can be accessed through modern web browsers.

1.5 Limitation

- Limited crop varieties for crop recommendation.
- Pest prediction uses a small dataset affecting accuracy.
- Prediction accuracy depends on correct user input.
- Disease detection requires clear, high-quality leaf images.
- No integration with IoT sensors for real-time field monitoring.
- Available only as a web application (no mobile app).
- Machine learning models require periodic retraining with updated datasets.
- Performance may vary when used in different agricultural regions or conditions.

1.6 Methodology

A systematic approach to software development, the Agile Method is a flexible, iterative method to software development and project management that prioritizes quick turnaround, client feedback, and teamwork. Agile promotes flexibility and ongoing improvement over the course of a project. This model will be used in the development of the Smart Agro System guaranteeing accurate specifications, multiple stages, and extensive testing prior to deployment.

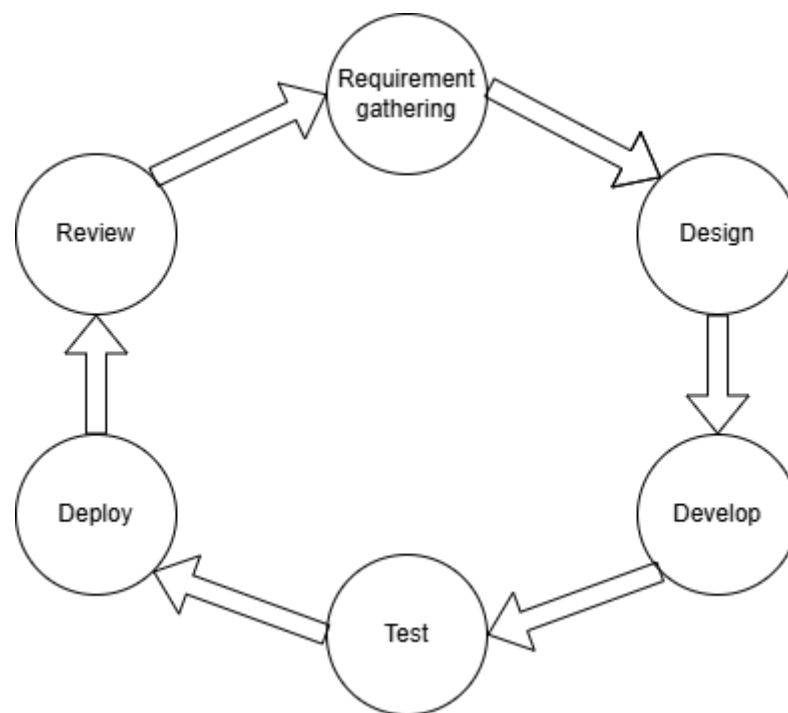


Figure 1.1 Agile Method

- **Gathering requirements:** This phase involves understanding what the users need and how the system should help them. Instead of defining every detail upfront, the team works in short, focused cycles to gather enough information to start building the system.
- **Design:** In this phase, the emotion based music recommendation system is designed based on the gathered requirements. This includes defining the user interface, database design, and other key system components.
- **Develop:** Work can begin once the team has defined the requirements. Designers and developers begin to work on their projects. This project aims at deploying a working product.
- **Test:** In this phase, testing the system to identify and finding errors are done. This phase helps to ensure the development meets the specified requirements as given requirement.

- **Deploy:** The emotion based music recommendation system is made available for usage following the ending of testing. This include setting up servers, allowing user access to the system, and ensuring an easy transition from testing to operation.
- **Review:** This phase in agile is when the client checks if the product is meeting their needs and gives continuous feedback if any changes are required.

1.7 Report Organization

This report is organized into five chapters.

Chapter 1 Introduction introduces the project, states its objectives, scope, and limitations, and describes the development methodology.

Chapter 2 Background Study and Literature Review presents the background study and a review of the related literature, covering Smart Agro, machine learning in agriculture, and the technical foundations of the system.

Chapter 3 System Analysis and Design describes the system analysis and design, including requirements analysis, UML modeling, database design, system architecture, and the algorithms used by the machine learning models.

Chapter 4 Implementation and Testing presents the implementation details of each module, the tools used, and the testing performed.

Chapter 5 Conclusion concludes the report, evaluates the outcomes, and recommends directions for future enhancement. A list of references and appendices follow the main chapters.

Chapter-2. Background Study and Literature Review

2.1 Background Study

The harvest sector remains one of the most important in the world, because it provides food security, work and economic stability. There are, however, several threats to the traditional farming means including unpredictable weather, erosion of soil, shortage of water and attacks by pests. The issues develop and tend to lead to poor utilization of resources and production of insufficient crops that impact the livelihood of the farmers.

As Smart farming or precision-based agriculture was introduced, technological solutions to this problem have been offered, including introducing data-driven models and machine learning algorithms into the agricultural activity. Smart Agro can also feed farmers with constantly updated data on soil conditions, temperature, humidity, and rainfall, enabling them to make well-informed decisions. Such systems could also be applied in proposing the most suitable crops to be grown under a particular environmental situation using historical and real time information so as to maximize the utilization of irrigation scheduling and minimize the utilization of fertilizers and pesticides.

Machine learning algorithms Random Forest and Gaussian Naive Bayes Classifier are significant to Smart Agro. Random Forest compares and contrasts the data of the soil and weather in order to provide sensible advice on the type of crop to be grown. Naive Bayes Classifier can be very useful to identify pests when dealing with image data of leaves, since it takes aspects like spots and discoloration to determine the probability of a particular disease to start an early treatment. MongoDB is integrated into a MERN stack and is used to manage the agricultural data, Node.js/Express.js is used to verify the server-side code and model, and React.js is used to provide the farmers with an intuitive interface that they can use to access these insights.

The background study indicates that modernization of the agriculture sector is necessary with the assistance of smart technologies, and indicates that data analytics, and machine learning can have a great influence on the crop yield, sustainability, and making a decision by a farmer. In the paper, it provides the foundation on the implementation of a Smart Agro System capable of providing precision-based crop recommendations.

2.2 Literature Review

Precision agriculture has evolved from basic mechanization to sophisticated AI-driven systems. The integration of IoT devices with farming equipment has enabled data collection at unprecedented scales, facilitating the development of decision support systems for farmers. The authors note that "Smart Agro technologies have the potential to increase crop yields by up to 25% while reducing resource consumption. [1]

Random Forest algorithm has been widely applied in agricultural recommendation systems due to its simplicity and effectiveness with spatial data. Random Forest based systems could achieve up to 92% accuracy in crop recommendation when trained on sufficient historical data encompassing soil properties, weather patterns, and yield records. [2]

The effectiveness of Random Forest in agricultural applications stems from its ability to handle multivariate data and provide recommendations based on similarity to known successful cases. Random Forest instance-based learning approach makes it particularly suitable for crop recommendation where historical successful patterns can guide future decisions". [3]

Naive Bayes classifiers have shown remarkable effectiveness in agricultural pest detection systems due to their efficiency with high-dimensional data and robustness against irrelevant features. Research demonstrated that a properly trained Naive Bayes classifier could achieve 89% accuracy in identifying common crop pests from image data. [4]

The algorithm's probabilistic foundation allows it to handle the uncertainty inherent in agricultural environments where multiple factors may contribute to pest outbreaks. As observed "Naive Bayes classifiers provide a computationally efficient method for early pest detection, which is crucial for implementing timely interventions". [5]

Despite the promising potential of Smart Agro systems, several implementation challenges persist. Rural connectivity issues, data privacy concerns, and the digital divide among farmers represent significant barriers to adoption. Research by Brown and Green (2021) highlighted that "while technology exists to revolutionize farming practices, the human factor remains the most significant challenge". [6]

7 Studies have consistently shown that well-implemented Smart Agro systems can deliver significant economic and environmental benefits. Research by Anderson and Martinez

(2020) demonstrated that farms using AI-driven recommendation systems reduced fertilizer use by 15-20% while maintaining or increasing yields. [7]

The environmental benefits extend beyond resource conservation to include reduced chemical runoff and lower greenhouse gas emissions. As noted by Thompson and Wilson (2021), "Precision agriculture technologies contribute to sustainable intensification by optimizing resource use and minimizing environmental impacts". [8]

The future of Smart farming systems lies in the integration of multiple technologies, including drones, robotics, and advanced analytics. Researchers predict that the next generation of farming assistants will feature enhanced predictive capabilities, autonomous decision-making, and seamless integration with farm management software. As observed by Chen et al. (2022), "The convergence of AI, IoT, and robotics will enable fully autonomous farming operations within the next decade, fundamentally transforming agricultural practices". [9]

Chapter-3. System Analysis and Design

3.1 System Analysis

Requirement analysis focuses on determining the needs of a system. The purpose of this analysis is to ensure that the software or system meets the desired goals. This system enables users to register their face, mark their attendance and view the attendance log as well.

3.1.1 Requirement Analysis

Functional requirement

User Authentication & Management:

- Farmers and admins can register/login (email/phone + password).
- Role-based access (e.g., farmers view recommendations; admins manage datasets).

Crop recommendation:

- Users input parameters (soil type, pH, rainfall, temperature).
- System processes data using Random Forest algorithm to suggest optimal crops.

Pest database and alert:

- Identifies the risk of pest in the field.

Plant disease detection:

- Identifies the plant disease

Use Case Diagram



Figure 3.1 Use Case Diagram

Non-functional requirement

- **Performance:** The system generates crop recommendations and updates pest alerts.
- **Scalability:** Designed to support many users with a modular Node.js backend for future expansion.
- **Usability:** Features a mobile-responsive interface and it is user friendly.
- **Reliability:** Ensures 99.9% uptime for key services and includes automated MongoDB backups to safeguard data.
- **Compatibility:** Compatible with major browsers (Chrome, Firefox, Safari)

3.1.2 Feasibility Study

Technical Feasibility

The MERN stack (MongoDB, Express.js, React.js and Node.js) is used to develop a fullstack web application. This stack is scalable, fast, and enables smooth development with JavaScript on both the frontend and the backend. Crop recommendation is done with the Random Forest algorithm, which is capable of classification problems related to inputs such as soil type, temperature and rainfall. Gaussian Naïve Bayes Classifier. Each of the chosen instruments and methods is possible and justified to be applied using the resources at hand.

Operational Feasibility

The Smart Agro Assistant is simplified and easy to use. It has a mobile-responsive interface that can work with smartphones and tablets, and can be used with different languages to support a wide range of users. The system provides simple to understand recommendations and alerts, and does not need any technical know-how to implement, thus easy to use by farmers in their daily lives.

Economic Feasibility

The open-source nature of the technologies used in this project, such as Node.js, MongoDB, and React, removes the cost of licensing altogether and keeps development costs to a minimum. The platform is non-technical and affordable, given that it is supported by the wider community and does not require special hardware to function. Its less costly financial investment is highly valued through increasing productivity and minimizing the loss of crops.

Schedule Feasibility

It is expected that the Smart Agro Assistant will be developed in one semester. To structure the schedule and separate the project into small parts, such as research, design, development, testing, and deployment, a Gantt chart is prepared. Every stage includes a 11 realistic timeframe, and the work is allocated to help maintain progress. By managing time adequately and having a development plan, the team can produce the project on time without affecting quality

3.1.3 Project Schedule

Table 3.1 Project Schedule

Task/Date	15 th April	24 th April	27 th May	2 nd Aug	15 th Sept
Planning					
Requirement gathering					
Sprint 1-Basic UI and DB Setup					
Testing Sprint 1					
Sprint 2- Crop and Pest analysis and disease recognition model					
Testing Sprint 2					
Sprint 3 - Crop and Pest analysis and disease recognition model enhancement					
Testing Sprint 3					
Sprint 4-Reports and UI Enhancement					
Testing Sprint 4					
Deployment					
Documentation					

The project schedule illustrates the planned activities and timeline for the development of the Smart Agro system. The project begins with the planning and requirement gathering

phases, followed by Sprint 1, which focuses on developing the basic user interface and database setup. Testing of Sprint 1 is then conducted to identify and resolve initial issues. Sprint 2 focuses on crop and pest analysis along with the implementation of the disease recognition model, followed by its testing phase. Sprint 3 involves enhancing the crop and pest analysis and disease recognition features, with subsequent testing to ensure their functionality and reliability. Sprint 4 focuses on report generation and user-interface enhancements, followed by final testing. After completing the development and testing activities, the system proceeds to the deployment phase. Documentation is carried out throughout the project period to record the development process, system design, implementation, testing, and final outcomes.

3.1.4 Object Modeling using object and class Diagram

Object modelling identifies the main classes, their attributes and methods, and their relationships:

- Farmer: represents a registered user with name, email, password, role, permissions override, profile picture, blocked status, farm name, location, and verification status.
- Record: represents a farming record of type crop, disease, or recommendation, storing the relevant parameters, the predicted outcome, the confidence, and associated medicines and purchased products.
- Product: represents a marketplace item with name, description, price, category, stock, seller, and approval status.
- Order: represents a purchase with buyer, items, total amount, shipping address, status, payment method, payment status, and transaction identifier.
- Medicine: represents the medicine database keyed by disease name, with chemical and organic medicine entries and suggested product names.
- Role: represents a named set of permissions with a description and a default flag.

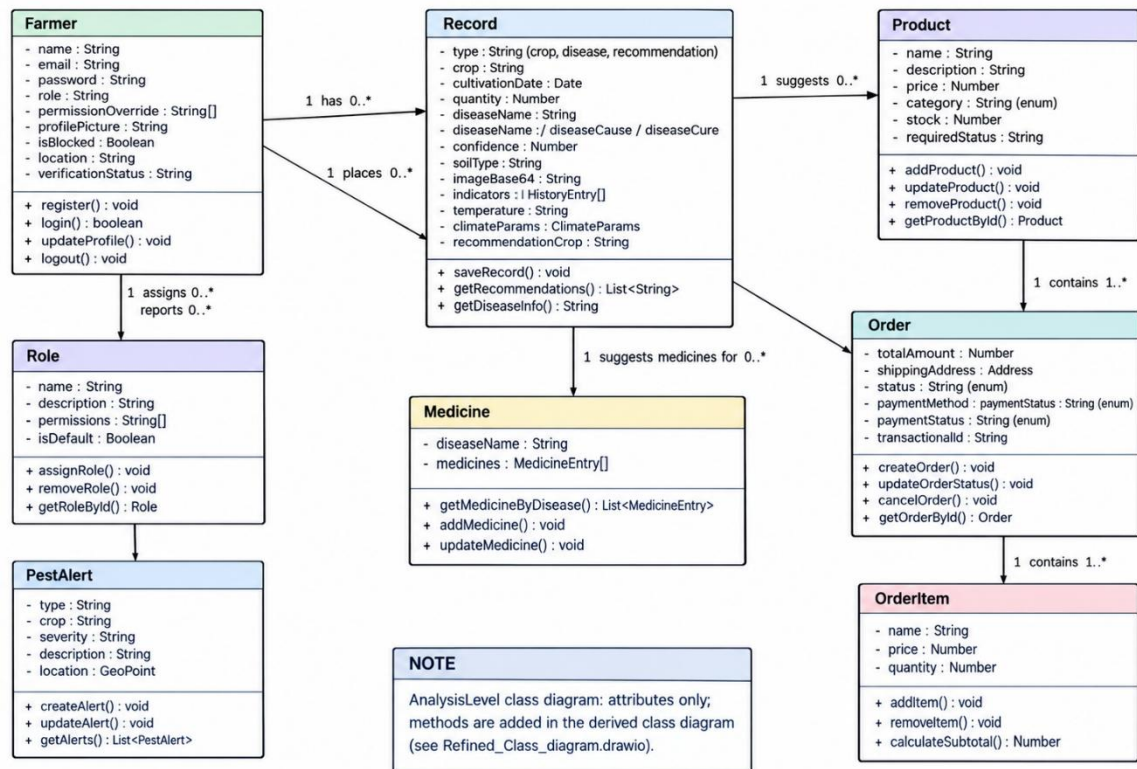


Figure 3.2 Class Diagram of Smart Agro

The class diagram represents the main entities of the Smart Agro System and their relationships. It shows how farmers create records, assign roles, receive pest alerts, and get recommendations for medicines and products. Products are connected to orders and order items, while the Record class stores crop, disease, soil, weather, and recommendation information used for analysis and decision-making.

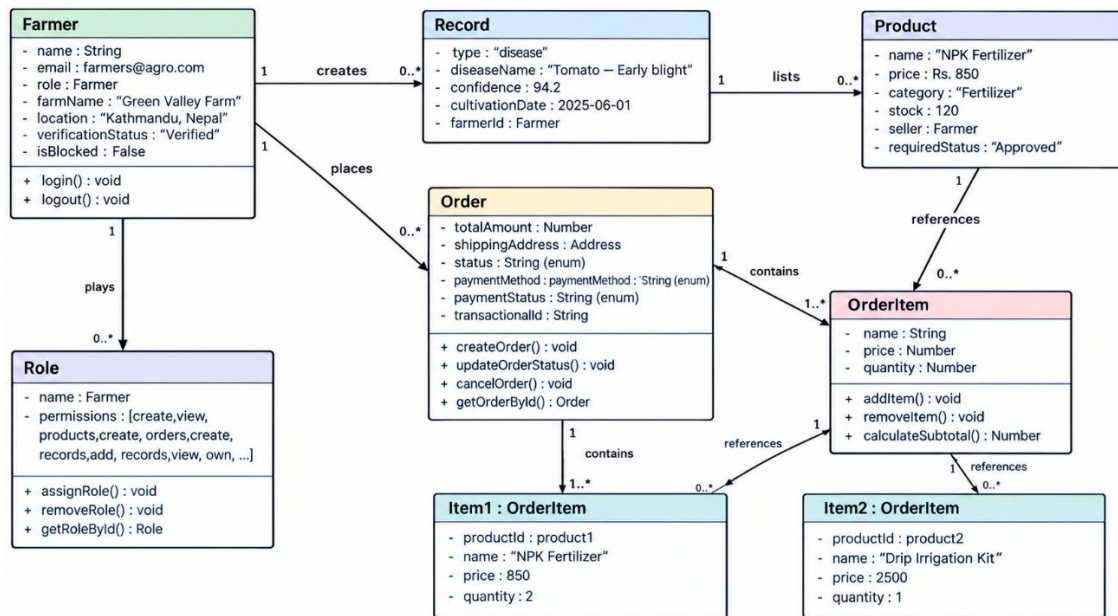


Figure 3.3 Object Diagram of Smart Agro

The object diagram shows the main relationships between farmers, records, products, orders, roles, and order items in the Smart Agro System. A farmer can create multiple records and place multiple orders, while each order contains one or more items that reference products. The diagram also shows how farmers are assigned roles and how records are associated with products and recommendations.

3.1.5 Dynamic Modeling using Sequence Diagram

Sequence diagrams model object interactions over time. It covers login and disease recognition: the AWT Controller returns a JWT the disease service returns class, cause, and confidence.

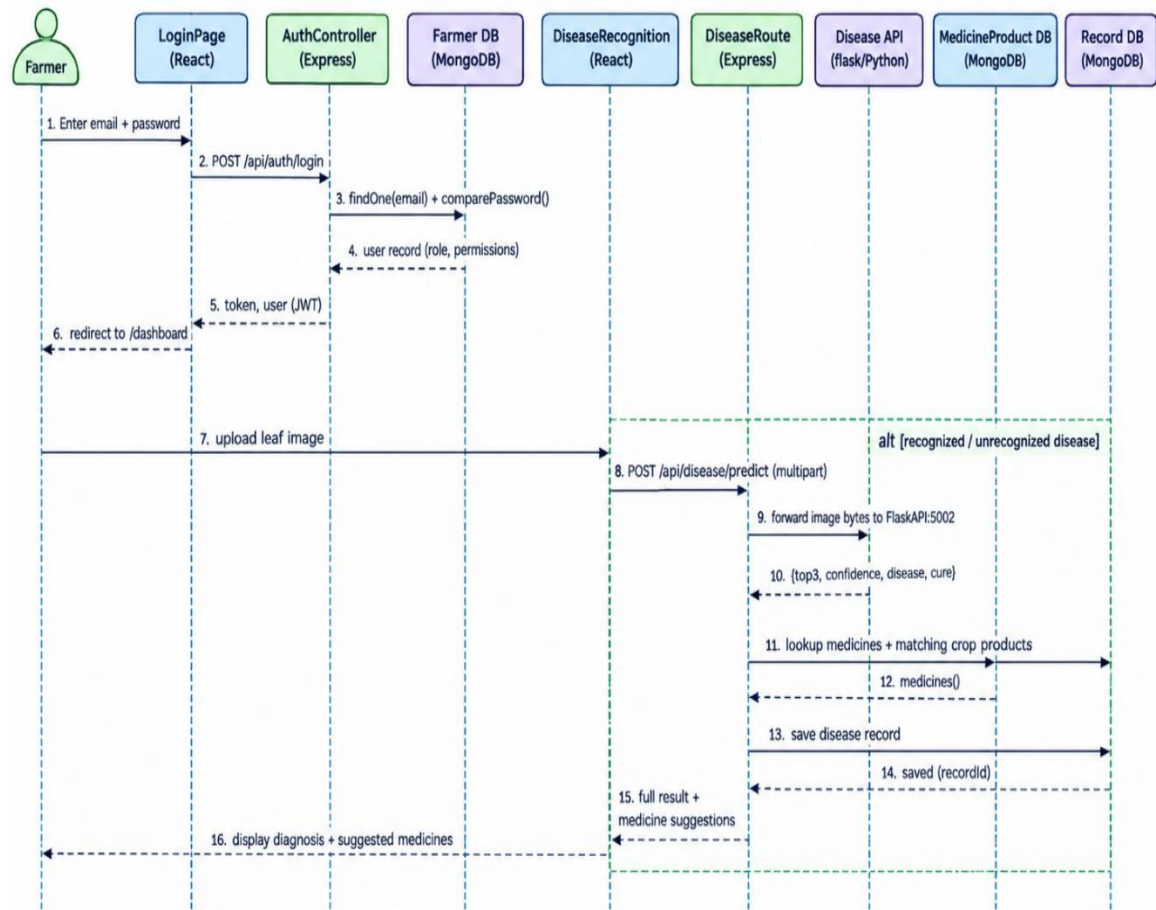


Figure 3.4 Sequence Diagram for User Flows

This sequence diagram illustrates the login and plant disease recognition process of the Smart Agro system. First, the farmer enters their email and password on the React login page, which sends the credentials to the Express authentication controller for verification against the Farmer database. After successful authentication, a JWT token is returned and the farmer is redirected to the dashboard. The farmer then uploads a leaf image, which is sent to the disease recognition module and forwarded to the Flask/Python Disease API for prediction. The API returns the top three possible diseases, confidence scores, disease information, and recommended cures. The system then retrieves suitable medicines and related products from the database and saves the diagnosis as a disease record. Finally, the complete diagnosis and suggested medicines are returned and displayed to the farmer.

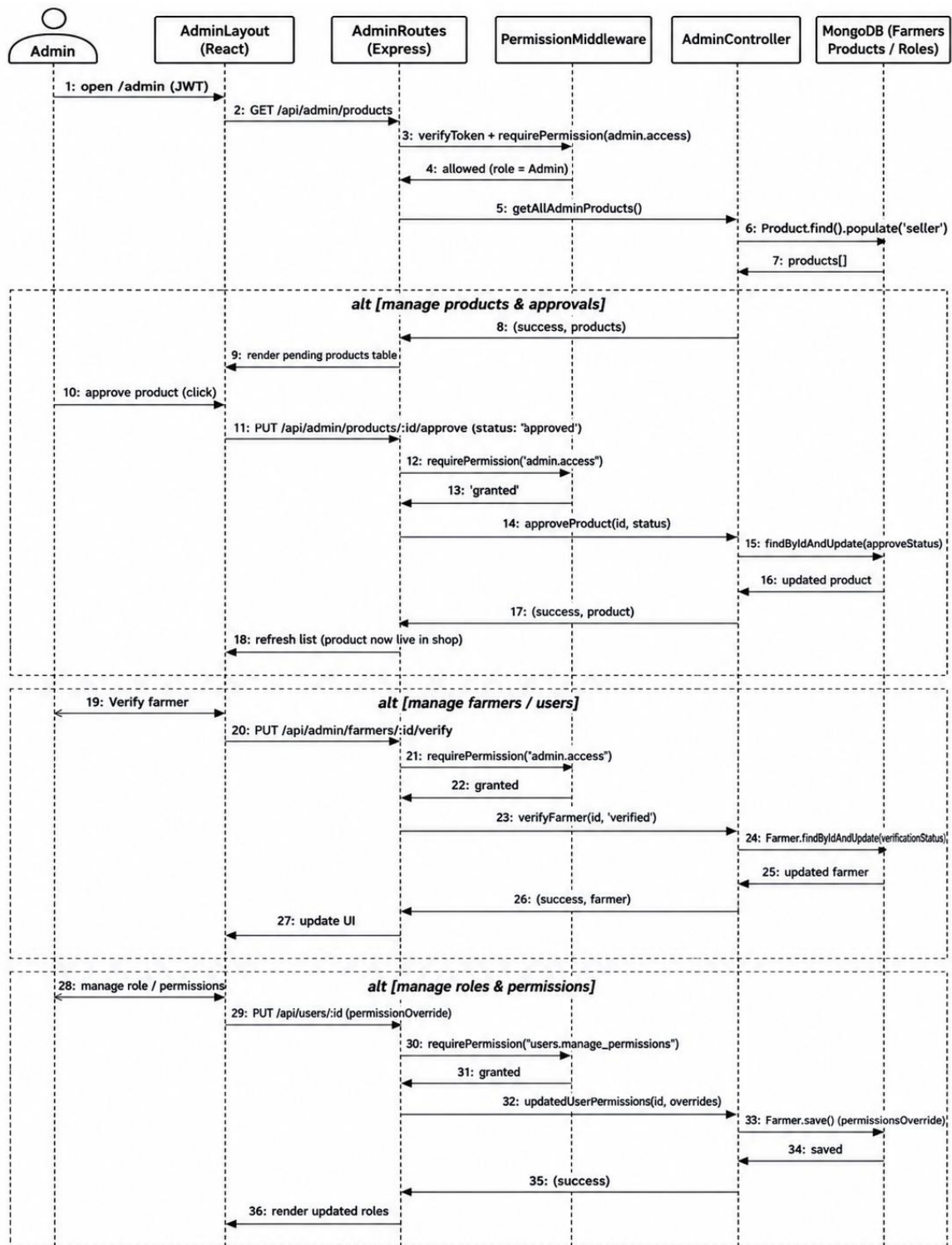


Figure 3.5 Sequence Diagram for Admin Management

This sequence diagram illustrates when the Admin accesses the portal, the React frontend requests product data from the Express backend, which verifies the Admin's JWT token and admin:access permission before fetching data from MongoDB to render the pending

products table. If the Admin approves a product or verifies a farmer account, the system sends PUT requests to the backend, verifying the `admin:access` permission each time before updating the respective approval or verification statuses in the database and refreshing the UI. Finally, when managing roles and permissions, the backend enforces a stricter `users:manage_permissions` requirement before saving any role overrides to the database and rendering the updated permissions on the frontend.

3.1.6 Process Modeling using Activity Diagram

Activity diagrams model the flow of control. This models the order process: the farmer checks out with a shipping address, and with eSewa the success page verifies the transaction and marks the order paid.

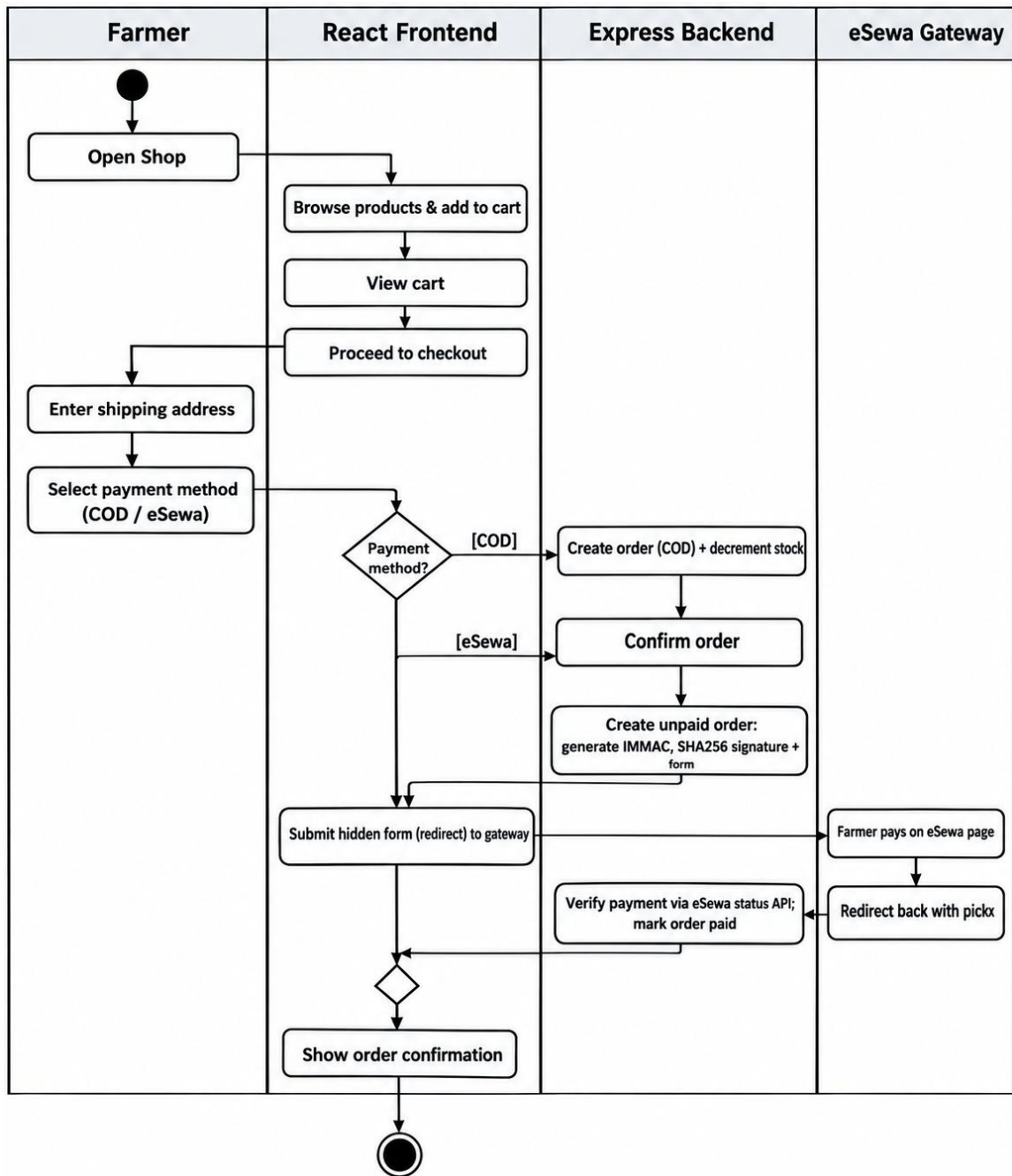


Figure 3.6 Activity Diagram for Order and Payment

he diagram illustrates the complete shopping and payment workflow of the system. The farmer opens the shop, browses products, adds items to the cart, and proceeds to checkout. After entering the shipping address, the farmer selects either COD or eSewa as the payment method. For COD, the backend creates the order and updates the stock, while for eSewa, the farmer is redirected to the eSewa gateway to complete the payment. The backend then verifies the payment status, updates the order, and finally displays the order confirmation

to the farmer. A useful next step would be to add this paragraph as the figure description in your report.

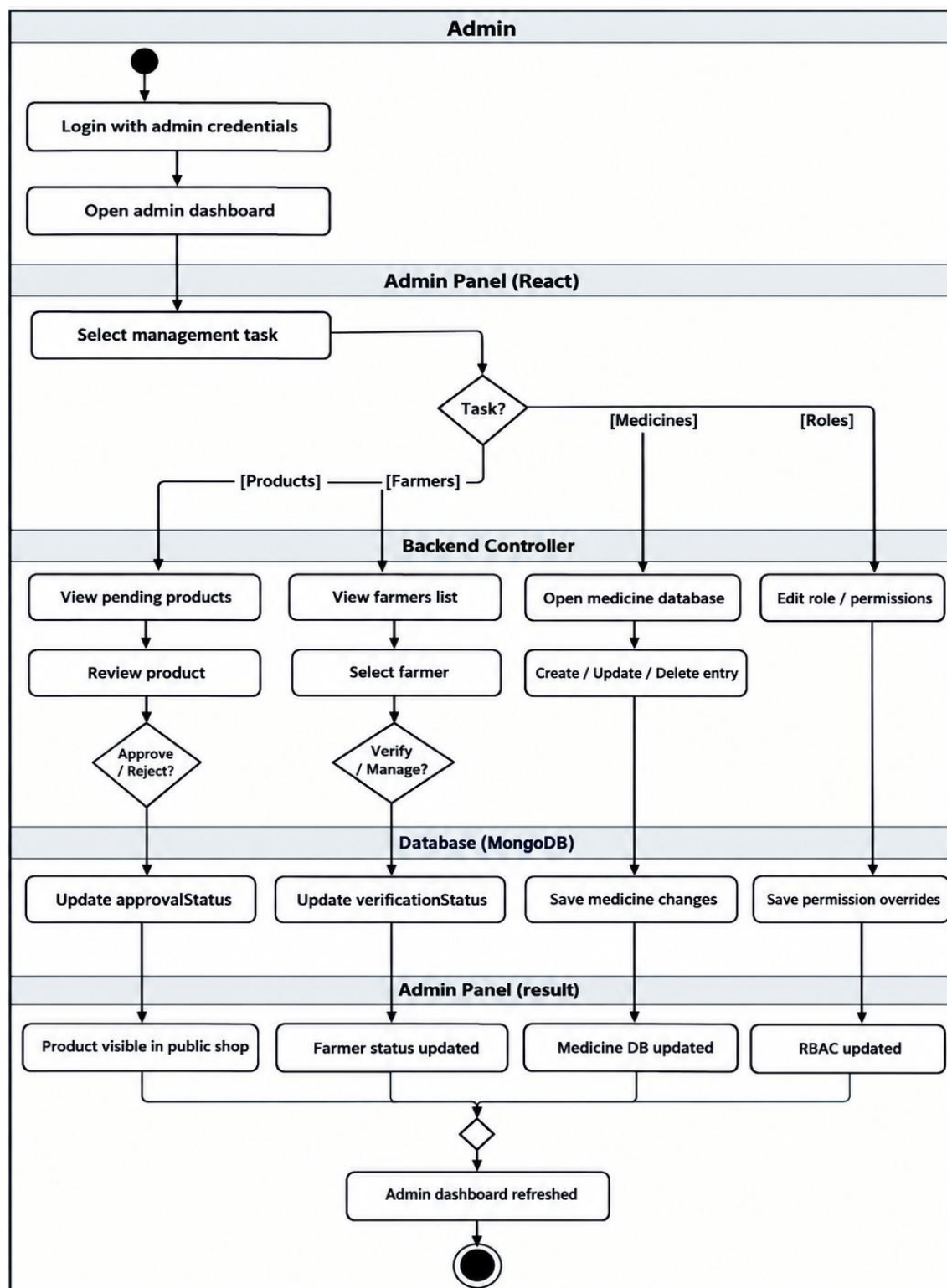


Figure 3.7 Activity Diagram for Admin Management

The diagram shows the admin management workflow, starting with admin login and access to the dashboard. The admin selects a task such as managing products, farmers, medicines, or roles and permissions. The backend processes the selected task and updates the MongoDB database accordingly. Finally, the updated information is displayed in the admin panel, and the dashboard is refreshed.

3.2 System Design

3.2.1 Refinement of Class Diagram

During design the analysis classes are refined into concrete classes following an MVC separation: Express controllers handle requests, Mongoose models handle persistence. it shows the refined diagram with comparePassword(), the authentication and permission middleware, and the ML service interfaces.

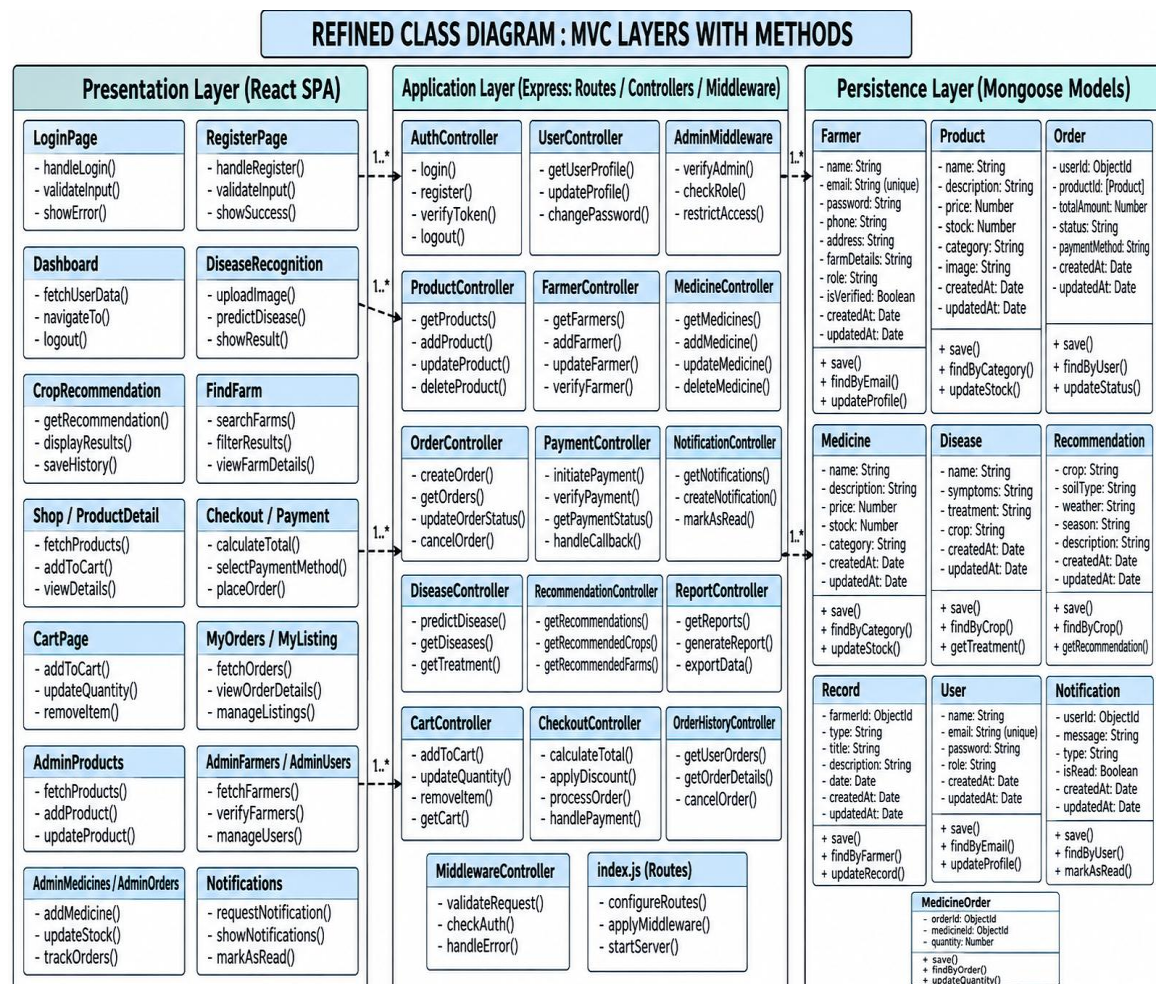


Figure 3.8 Refined Class Diagram (MVC Layers)

The diagram presents the system architecture using the MVC pattern with three main layers: the React presentation layer, the Express application layer, and the Mongoose persistence layer. The React layer contains pages and user interfaces, the Express layer manages controllers, routes, authentication, payments, and business logic, while the Mongoose layer stores and manages data such as farmers, products, orders, users, medicines, and notifications. The connections between these layers show how user requests are processed and how data is stored and retrieved from the database.

3.2.2 Component Diagram

The component diagram describes the major components and their interactions. Figure 3.12 shows the React SPA, the Express backend with RBAC, MongoDB, the FastAPI services, and the eSewa gateway;

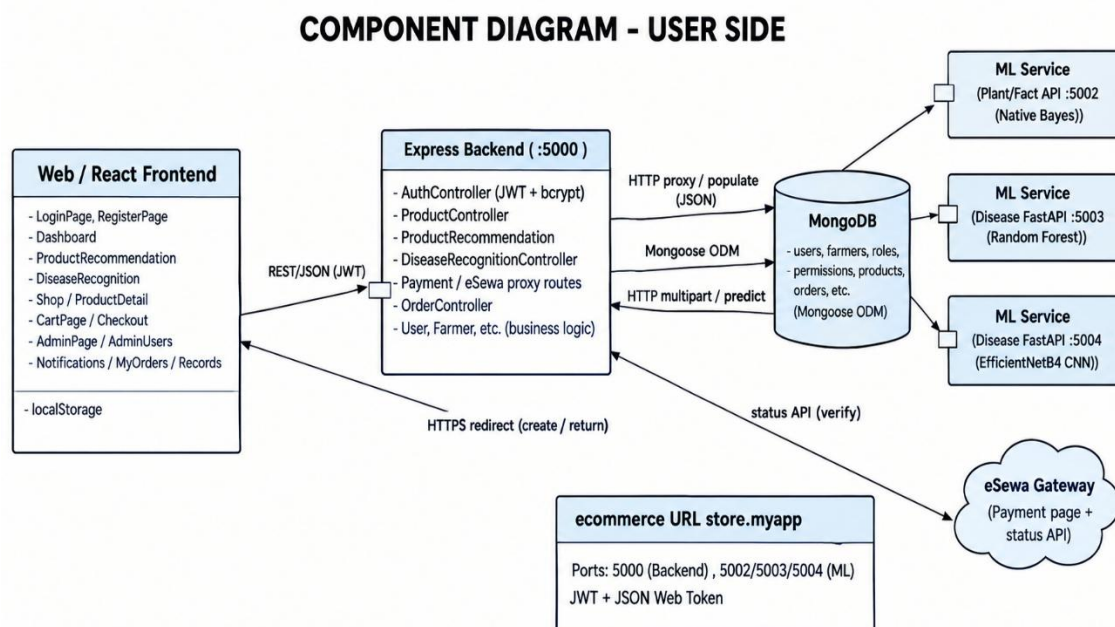


Figure 3.9 Component Diagram - User Side

The component diagram shows how the main parts of the user-side system communicate with each other. The React frontend sends requests to the Express backend, which handles authentication, products, orders, payments, and other business logic. The backend connects to MongoDB for storing and retrieving data and communicates with machine learning services for plant, crop, and disease-related predictions. It also connects to the eSewa gateway to process and verify online payments, allowing the complete system to work together efficiently.

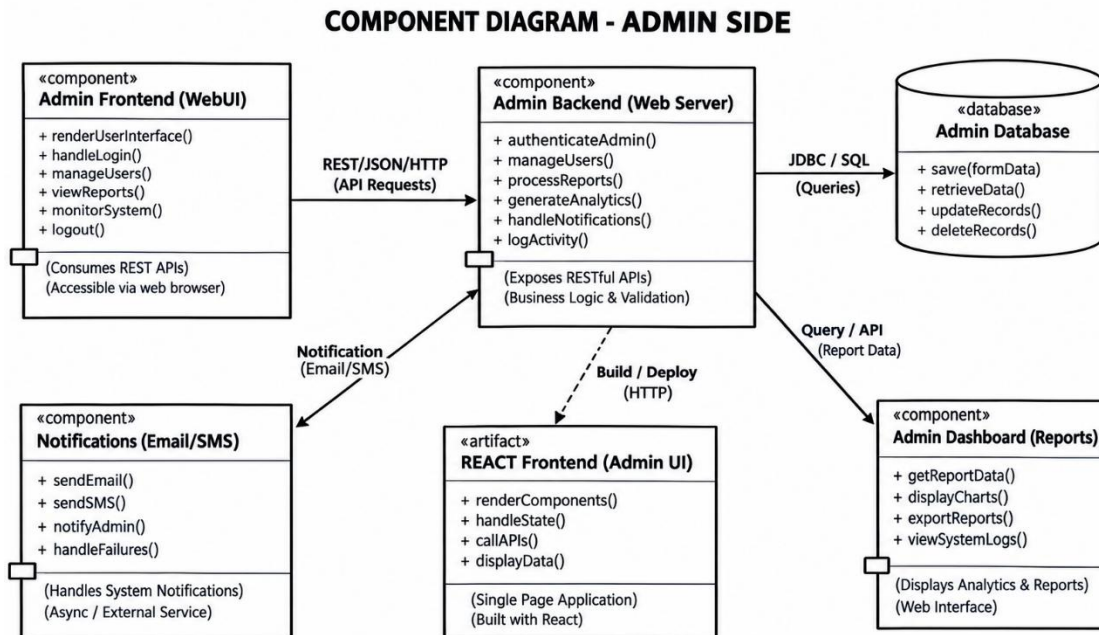


Figure 3.10 Component Diagram - Admin Side

The component diagram shows how the admin frontend, backend, database, dashboard, notifications, and React UI interact with each other. The frontend communicates with the backend through REST/JSON/HTTP, while the backend manages authentication, users, reports, analytics, and notifications. The backend connects to the database for storing and retrieving data, sends notifications through email/SMS services, and provides report data to the admin dashboard.

3.2.3 Deployment Diagram

In the local deployment all components run on one machine. The React dev server on port 3000, the Express backend on port 5000, the services on ports 5002-5004, and MongoDB.

DEPLOYMENT DIAGRAM - SMART AGRO SYSTEM (Local deployment)

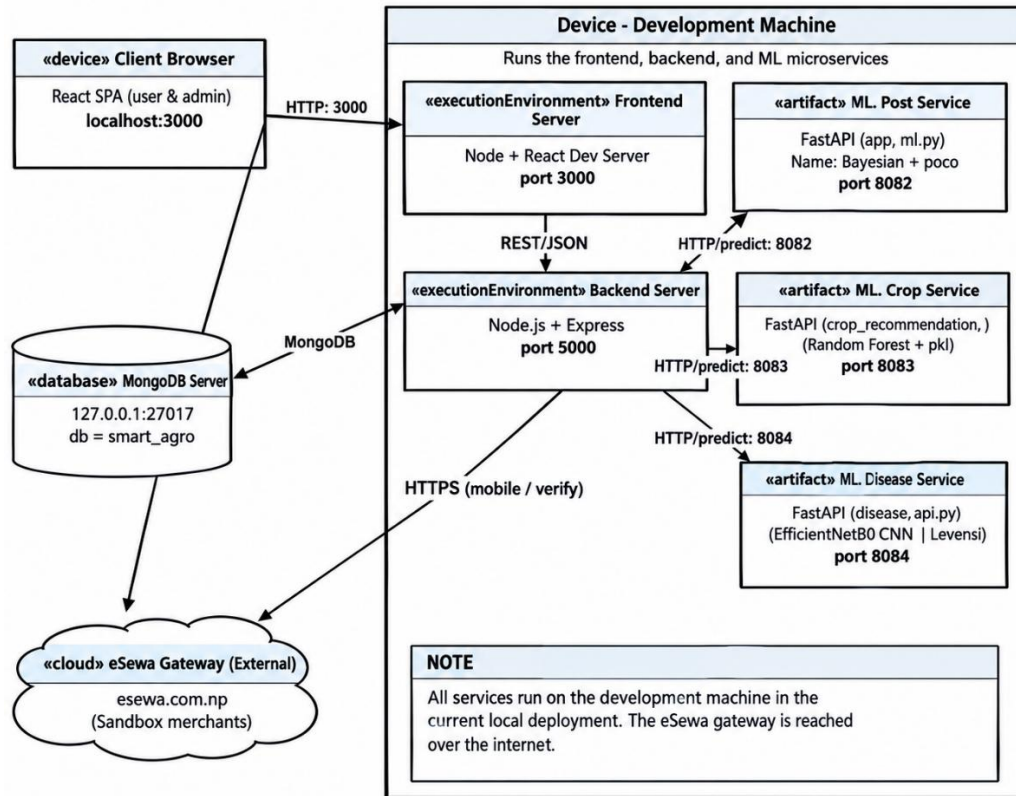


Figure 3.11 Deployment Diagram (Local Deployment)

The deployment diagram shows how the Smart Agro System is deployed on a local development machine. The client browser accesses the React frontend through port 3000, which communicates with the Node.js/Express backend on port 5000. The backend connects to MongoDB for data storage and communicates with separate ML services for post prediction, crop recommendation, and disease detection, while eSewa is used as an external payment gateway.

3.2.4 Wireframes

The wireframe shows a login page with a central container. At the top, the word "Login" is centered. Below it is a large rectangular box containing the text "SMART AGRO". Underneath this box are two input fields: the first is labeled "Email" and contains the placeholder text "name@example.com"; the second is labeled "Password" and contains seven asterisks. Below the password field is a button labeled "Login". At the bottom of the container, there is a link that says "Don't have an account? Register".

Figure 3.12 Wireframe Diagram of Login Page

This Wireframe Diagram shows the design of login page of Smart Agro.

The wireframe shows a shop page layout. At the top, there is a navigation bar with a list of categories: "[categories: All | Fertilizer | Pesticide | Tool | Seed]". To the right of the navigation bar is a button labeled "Cart (2)". Below the navigation bar is a grid of six product cards arranged in two rows of three. Each card contains a placeholder for an image "[img]", the product name, and the price. The products and prices are: NPK Fertilizer (Rs. 850), Drip Kit (Rs. 2,500), Pesticide (Rs. 1,200) in the first row; Seed Pack (Rs. 400), Sprayer (Rs. 3,000), and Compost (Rs. 750) in the second row.

Figure 3.13 Wireframe Diagram of Shop Page

This Wireframe Diagram shows the design of shop page of Smart Agro.

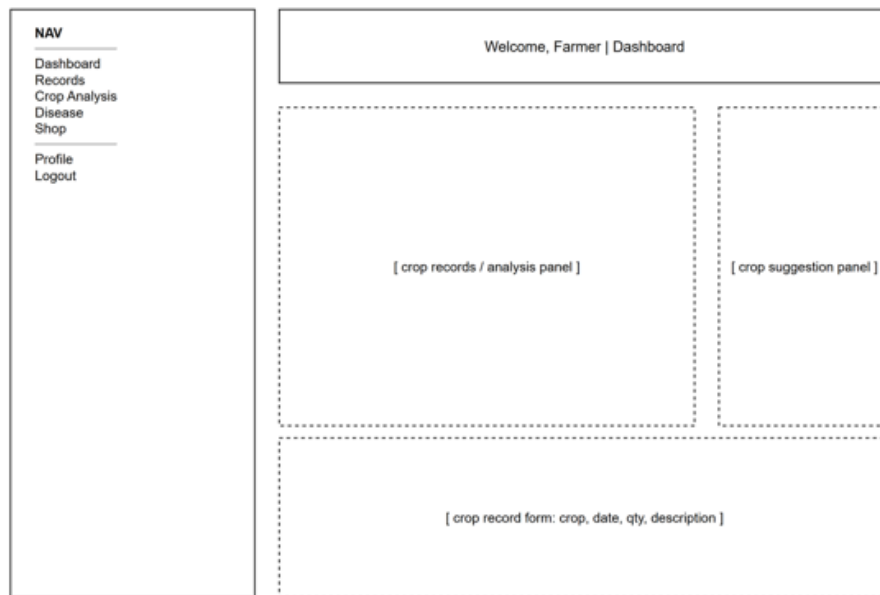


Figure 3.14 Wireframe Diagram of User Dashboard Page

This Wireframe Diagram shows the design of user dashboard page of Smart Agro.

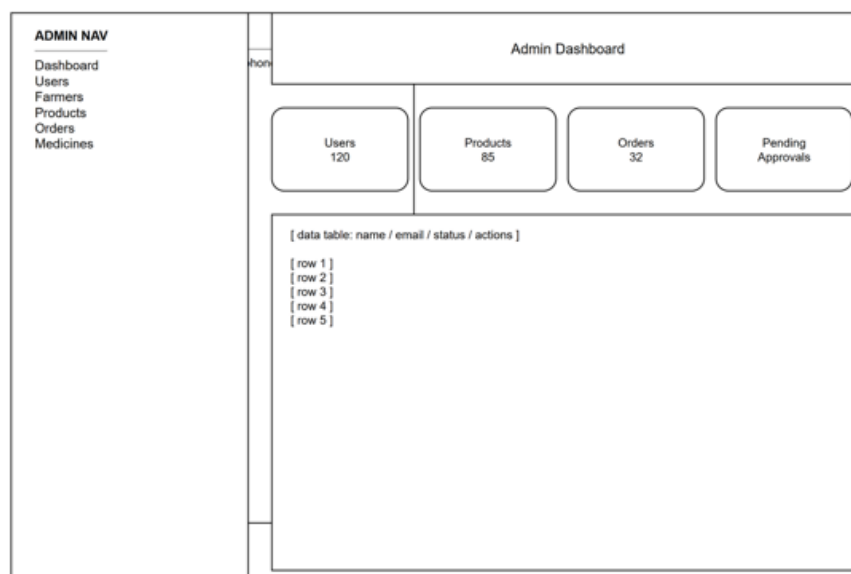


Figure 3.15 Wireframe Diagram of Admin Dashboard Page

This Wireframe Diagram shows the design of admin Dashboard page of Smart Agro.

NAV
Dashboard
Records
Crop Recommendation
Pest Risk
Disease
Shop
Profile
Logout

Crop Recommendation

Soil & Climate Parameters

Nitrogen (N)
[45]

Phosphorous (P)
[18]

Potassium (K)
[30]

Temperature (degC)
[26.5]

Humidity (%)
[72]

pH
[6.4]

Rainfall (mm)
[210.7]

Predict Crop

Recommended Crop: Maize

Confidence: 99.1%

Top 5 candidates:

1. Maize - 99.1%
2. Chickpea - 0.3%
3. Pigeonpea - 0.2%
4. Mothbeans - 0.2%
5. Jute - 0.1%

Growing tips:

Plant after last frost; full sun...

Similar-condition crops:

1. Rice (dist. 12.4)
2. Banana (dist. 18.7)

Figure 3.16 Wireframe Diagram of Crop Recommendation Page

This Wireframe Diagram shows the design of crop recommendation page of Smart Agro.

NAV
Dashboard
Records
Crop Recommendation
Pest Risk
Disease
Shop
Profile
Logout

Disease Recognition

Upload Leaf Image

[leaf image preview / drag & drop]

Choose Image

[jpeg / png, max 10 MB]

Result

Recognized: Tomato__Late_blight

Confidence: 96.4%

Top 3 predictions:

1. Tomato__Late_blight - 96.4%
2. Tomato__Early_blight - 2.1%
3. Tomato__healthy - 1.5%

Cause: Phytophthora infestans...

Cure: Apply copper-based fungicide...

Matching medicines:

1. Fungicide X (Rs. 450)
2. Fungicide Y (Rs. 380)

Save Record

View Related Products

[confidence < 0.5 => Unrecognized]

Figure 3.17 Wireframe Diagram of Disease Recognition Page

This Wireframe Diagram shows the design of disease recognition page of Smart Agro.

3.2.5 Interface Design (UI Interface/Interface Structure Diagrams)

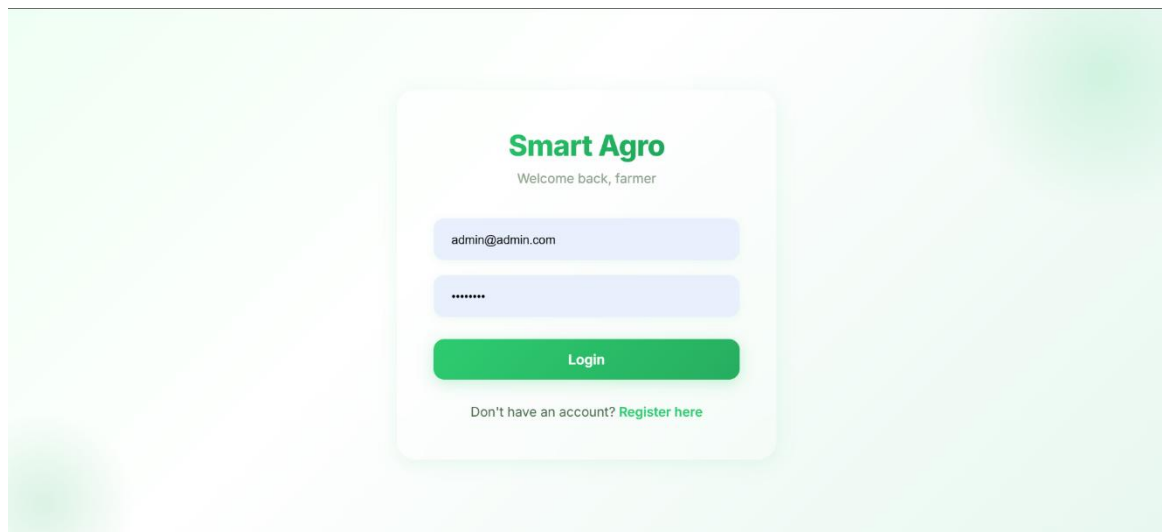


Figure 3.18 Interface Design of Login Page

This is the final design of login page based on wireframe diagram.

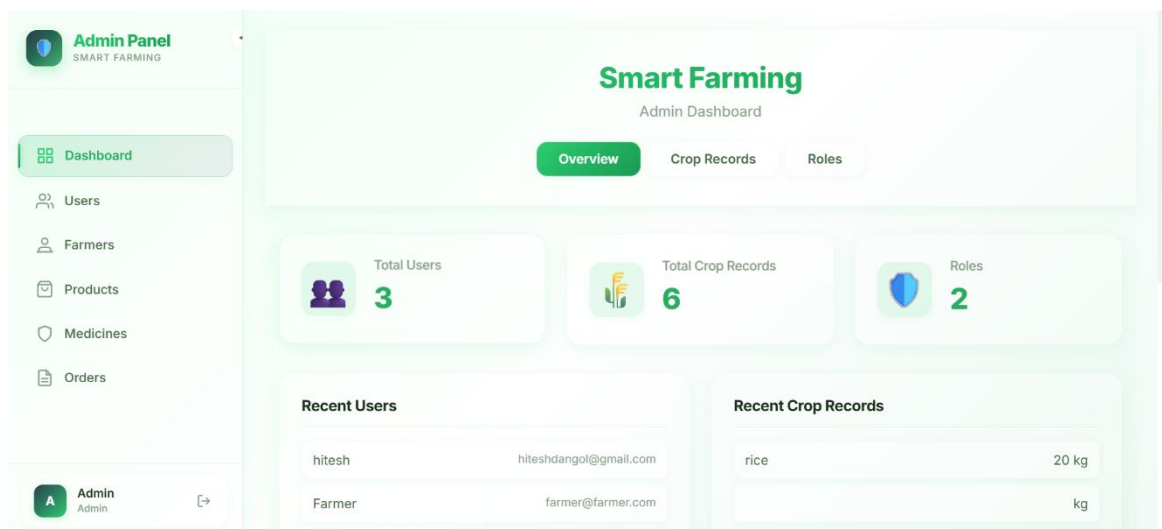


Figure 3.19 Interface Design of Admin Dashboard Page

This is the final design of admin dashboard page based on wireframe diagram.

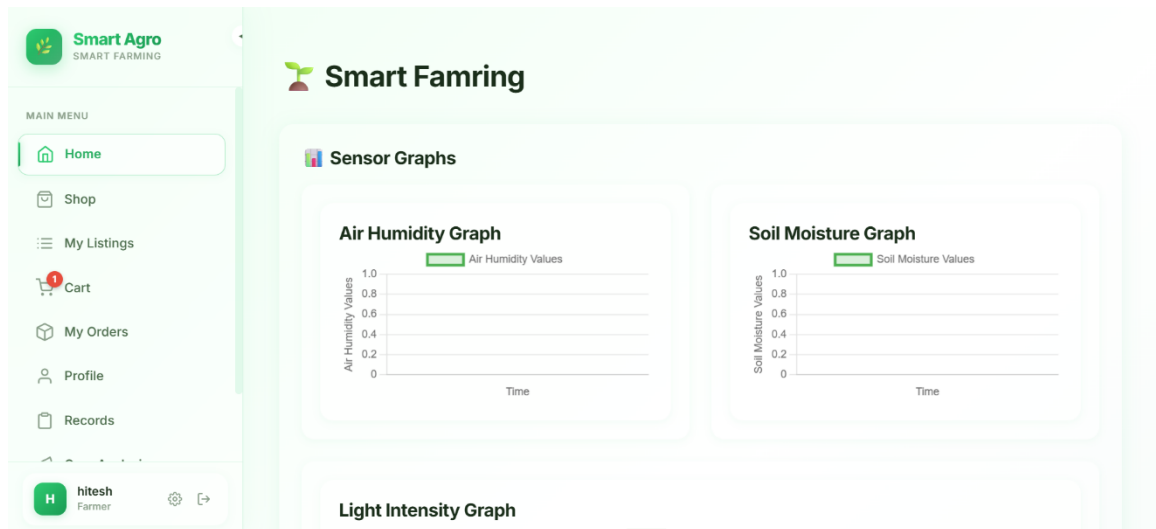


Figure 3.20 Interface Design of User Dashboard Page

This is the final design of user dashboard page based on wireframe diagram.

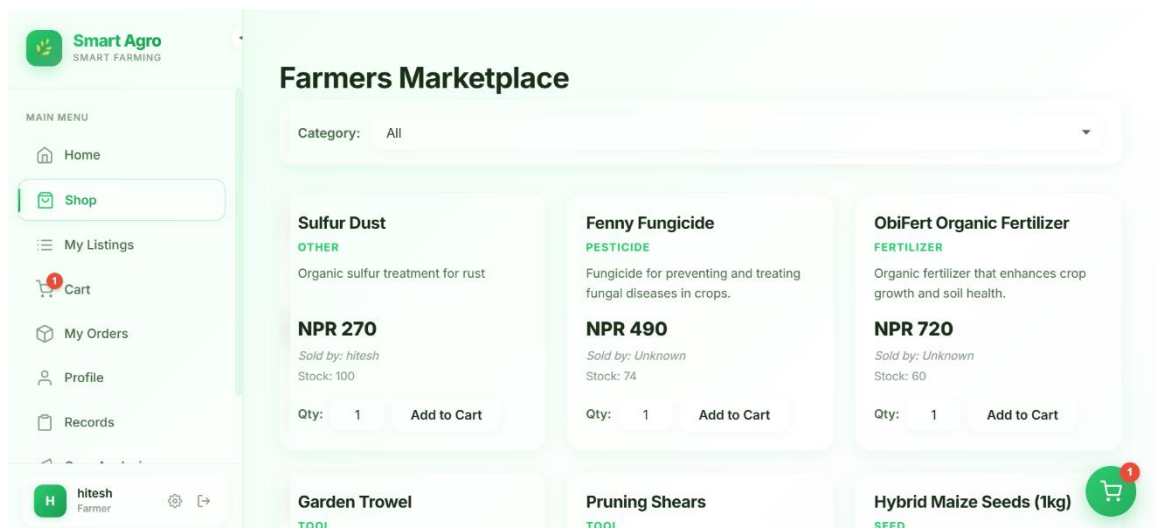


Figure 3.21 Interface Design of Shop Page

This is the final design of shop page based on wireframe diagram.

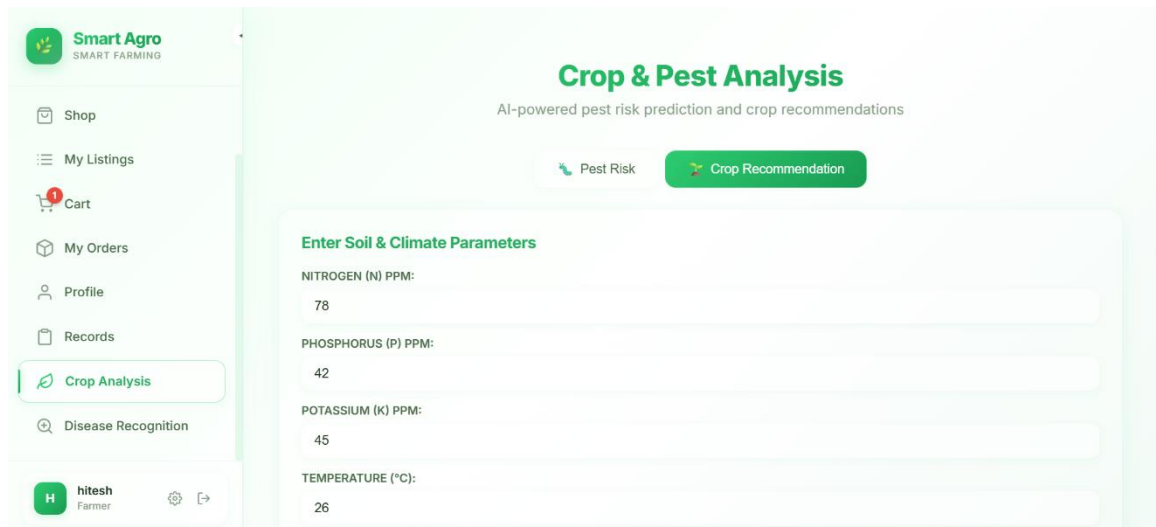


Figure 3.22 Interface Design of Crop Recommendation Page

This is the final design of crop Recommendation page based on wireframe diagram.

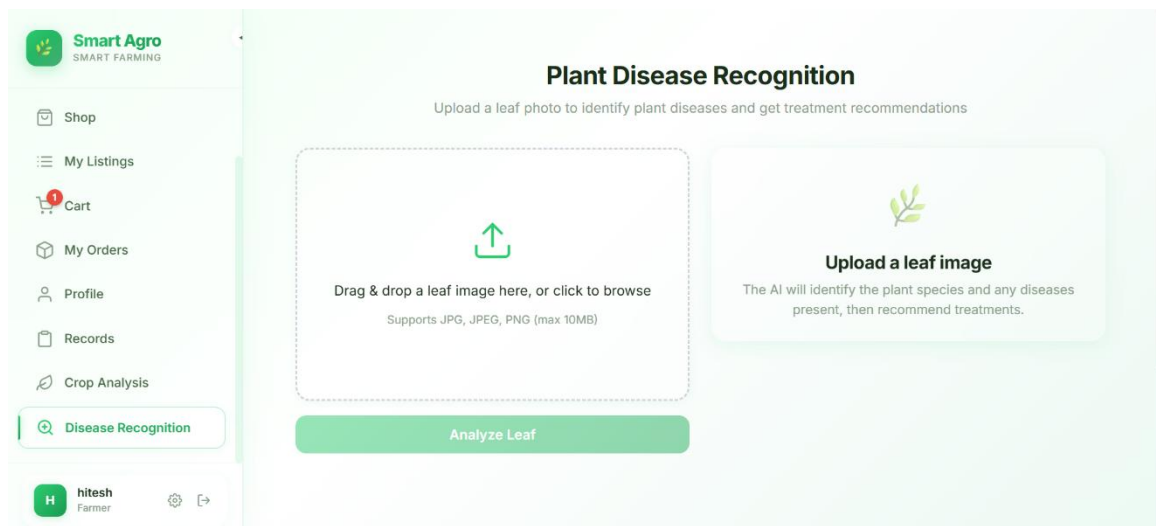


Figure 3.23 Interface Design of Plant Disease Recognition Page

This is the final design of Plant Disease Recognition page based on wireframe diagram.

3.3 Algorithm

Random Forest Algorithm

Random Forest is a powerful ensemble machine learning algorithm primarily used for classification and regression tasks. It combines the predictions of multiple decision trees to improve accuracy. In crop recommendation, Random Forest can predict the most suitable crop based on soil and environmental features like nitrogen (N), phosphorus (P), potassium

(K), temperature, humidity, pH, and rainfall. Theory of Random Forest Random Forest combines multiple decision trees to improve prediction accuracy and reduce overfitting.

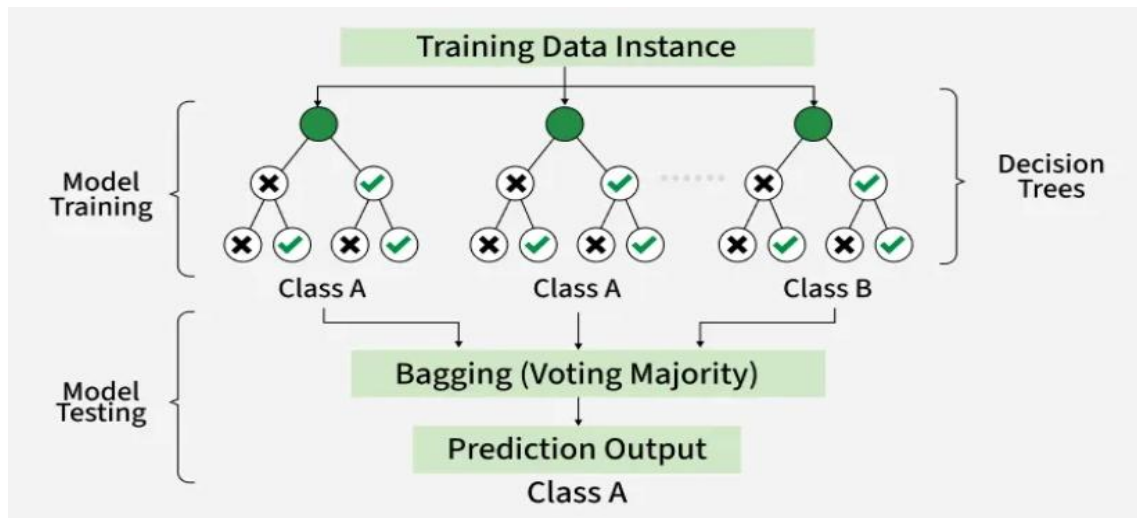


Figure 3.24 Working Mechanism of Random Forest Algorithm

$$\hat{y} = \text{mode}\{T_1(x), T_2(x), \dots, T_n(x)\}$$

$$P(y = c | x) = \frac{1}{n} \sum_{i=1}^n I(T_i(x) = c)$$

Workflow The Random Forest algorithm begins with the preparation of the dataset, which includes features such as soil nutrients (N, P, K), temperature, humidity, pH, and rainfall, along with the target label representing the crop type. The algorithm constructs multiple decision trees using bootstrap sampling, where each tree is trained on a randomly selected subset of the data with replacement. At each node of a tree, a random subset of features is chosen to determine the best split, often based on criteria like Gini impurity or entropy. Each tree grows independently to its full depth or until a stopping condition is met, producing diverse models that capture different patterns in the data. For predicting the optimal crop for new input conditions, all trees in the forest make individual predictions, and the final output is determined by majority voting, ensuring robustness and reducing overfitting compared to a single decision tree. The model's performance is evaluated using metrics such as accuracy, confusion matrix, and classification reports, and feature importance can be analyzed to understand the influence of each factor on crop selection. This workflow allows the Random Forest model to provide reliable crop recommendations under varying soil and environmental conditions.

Gaussian Naïve Bayes algorithm

Gaussian Naive Bayes algorithm is a probabilistic symptom checker of a plant health. It estimates the probability that a specific combination of observed symptoms (e.g., leaf spots, discoloration), was caused by a specific pest or disease. Its naive assumption of independent features is at best a simplification, but it is strikingly successful in this exercise. It is an effective method of giving a quick diagnosis of a crop, thereby helping to prevent crop loss by providing early intervention and specific treatment by analyzing historical data on symptom-pest correlations. The Core Idea: Gaussian Bayes' Theorem

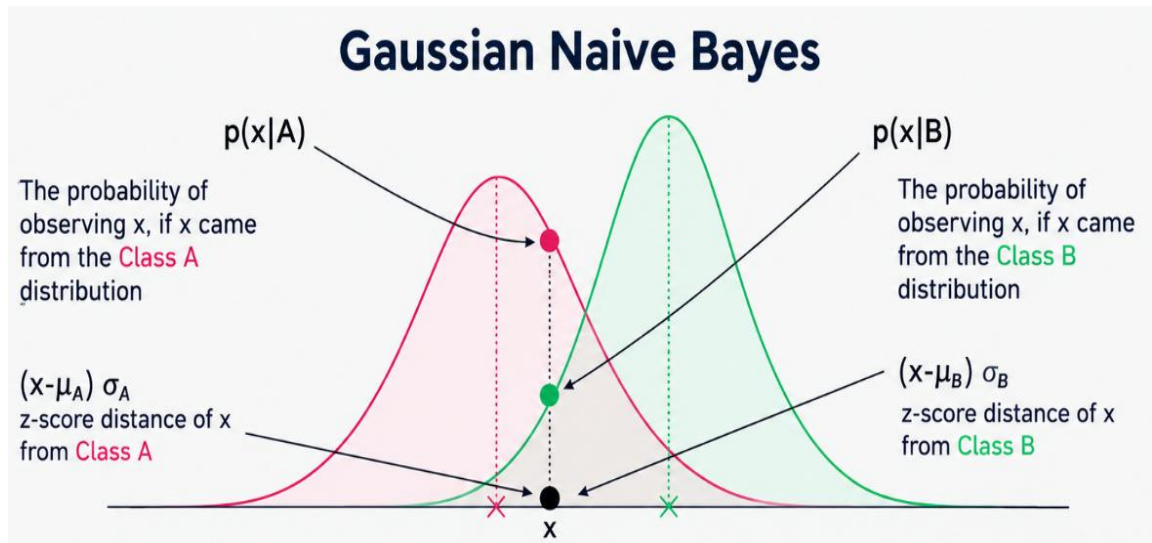


Figure 3.25 Working Mechanism of Gaussian Naive Bayes Algorithm

$$P(C_k | X) = \frac{P(X | C_k)P(C_k)}{P(X)}$$
$$P(x_i | C_k) = \frac{1}{\sqrt{2\pi\sigma_{ki}^2}} \exp\left(-\frac{(x_i - \mu_{ki})^2}{2\sigma_{ki}^2}\right)$$
$$P(X | C_k) = \prod_{i=1}^d P(x_i | C_k)$$
$$\hat{C} = \operatorname{argmax}_k P(C_k) \prod_{i=1}^d P(x_i | C_k)$$

Convolutional Neural Network (CNN) algorithm

A Convolutional Neural Network (CNN) is a deep learning model that is widely used for image classification and computer vision tasks. CNNs automatically learn important visual features, such as edges, textures, shapes, and patterns, directly from images without

requiring manual feature extraction. This makes them highly effective for identifying and classifying plant diseases from leaf images.

In the Smart Agro System, the disease recognition module utilizes a fine-tuned EfficientNetB4, which is a state-of-the-art CNN architecture. The model processes plant leaf images of 160×160 pixels and classifies them into 39 disease categories. It returns the top three predicted disease classes along with their confidence scores. To improve prediction reliability, a confidence threshold of 0.5 is applied, and images with confidence below this threshold are labeled as "Unrecognized."

The use of CNN technology enables the system to detect plant diseases accurately by learning complex visual patterns from large image datasets. This assists farmers in identifying diseases at an early stage and obtaining timely treatment recommendations, thereby reducing crop losses and improving agricultural productivity.

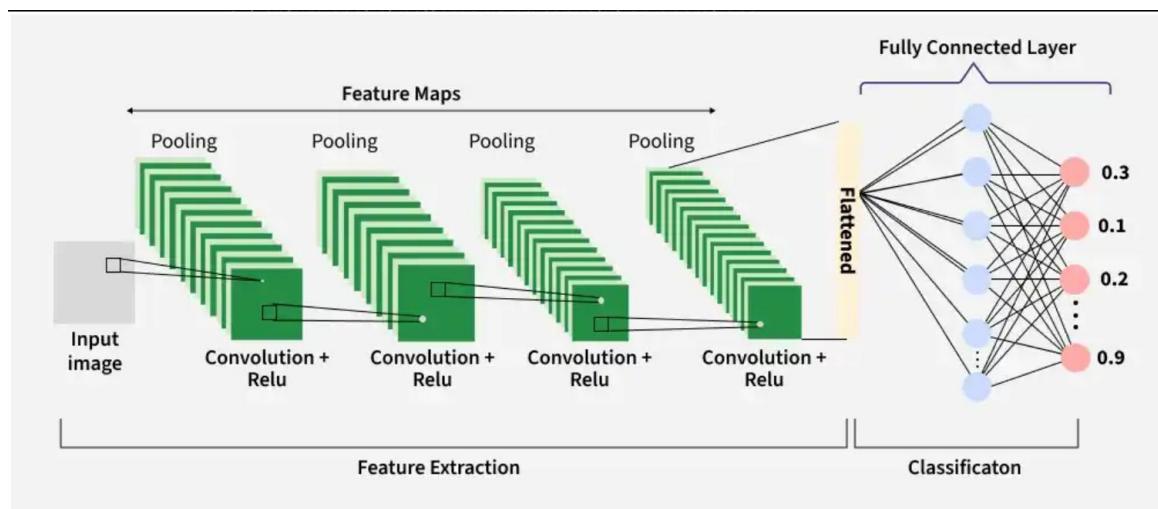


Figure 3.26 Working Mechanism of CNN Algorithm

$$Z_{i,j} = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} X_{i+m,j+n} K_{m,n} + b$$

$$A_{i,j} = \max(0, Z_{i,j})$$

Chapter-4. Implementaion and Testing

4.1 Tools Used

React Js

React JS is the primary frontend library used to build the user interface of the platform. It enables the development of dynamic, component-based web applications, allowing for efficient rendering and state management. React Router is integrated for seamless navigation between pages, and Material-UI provides a set of pre-built UI components and icons for a modern look and feel. Custom CSS is used to further style and layout the application, ensuring responsiveness and accessibility.

MongoDB

MongoDB serves as the main database for the project. It is a NoSQL, document-oriented database that stores data in flexible JSON-like documents. This flexibility allows for rapid development and easy scaling. Mongoose is used as the Object Data Modeling (ODM) library, providing schema definitions, validation, and query capabilities, which help maintain data integrity and simplify interactions with the database.

Node JS

Node JS is the backend runtime environment, enabling server-side JavaScript execution. Express JS is used as the web framework to build RESTful APIs, handle routing, and manage middleware. The backend is responsible for processing requests, handling authentication (using JWT and bcrypt), managing business logic, and interacting with MongoDB to store and retrieve data.

GitHub

GitHub is a web-based version control repository and Internet hosting service. It is mostly used for code. It offers all of the distributed version control and source code management (SCM) functionality of Git as well as adding its own features. GitHub offers both plans for private and free repositories on the same account which are commonly used to host open source software projects. Projects on GitHub can be accessed and manipulated using the standard Git command-line interface and all of the standard Git commands work with it. GitHub also allows registered and non-registered users to browse public repositories on the site. Multiple desktop clients and Git plugins have also been created by GitHub and other third parties that integrate with the platform.

MS Word:

MS word is one of the powerful tools for creating, editing, formatting, and printing documents. Here, in this project MS Word was used for the documentation of the project which contains the function like page layout, tables and columns, templates, images and so on.

Draw.io:

Draw.io is a popular one diagramming and flowcharting tool. Using its interface all of the diagrams were created for the system. All of the diagrams were created using this tool. We don't need to install any of the software for accessing it.

4.2 Implement Details of the modules

User Registration Module

```
exports.registerFarmer = async (req, res) => {
  const { name, email, password, role } = req.body;

  if (await Farmer.findOne({ email: email.toLowerCase() })) {
    return res.status(400).json({ message: "Farmer already exists" });
  }

  const farmer = await Farmer.create({
    name,
    email: email.toLowerCase(),
    password,
    role
  });

  res.status(201).json({
    token: generateToken(farmer._id)
  });
};
```

Listing 4.1 Code Snippet of User Registraton

The registration controller validates user input, checks for duplicate email addresses, assigns a default role, stores the user information in the database, and generates a JWT after successful registration.

User Login Module

```
exports.loginFarmer = async (req, res) => {
  const farmer = await Farmer.findOne({ email }).select("+password");

  if (!farmer || !(await farmer.comparePassword(password))) {
    return res.status(401).json({ message: "Invalid credentials" });
  }

  res.json({
    token: generateToken(farmer._id)
  });
};
```

Listing 4.2 Code Snippet of User Login

During login, the entered password is compared with the encrypted password stored in the database. If the credentials are valid, a JWT access token is generated and returned.

Plant Disease Recognition Module

```
def extract_features(image_bytes):
    img = tf.keras.utils.load_img(image_bytes)
    feature = tf.keras.utils.img_to_array(img)
    feature = tf.image.resize(feature, (160, 160))
    return np.array([feature])

probs = model.predict(img, verbose=0)[0]
top3_idx = np.argsort(probs)[-3:][::-1]
```

Listing 4.3 Code Snippet of Plant Disease Recognition

The disease recognition module is implemented using a fine-tuned EfficientNetB4 Convolutional Neural Network (CNN). Uploaded plant leaf images are preprocessed by resizing them to 160×160 pixels before being passed to the trained model for prediction. The system returns the top three predicted disease classes with their confidence scores and provides the corresponding disease cause and treatment information.

Crop Recommendation Module

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

model.fit(X_train_scaled, y_train)
```

Listing 4.4 Code Snippet of Crop Recommendation

The crop recommendation module uses a Random Forest classifier trained on soil nutrients and environmental parameters, including Nitrogen (N), Phosphorus (P), Potassium (K), temperature, humidity, pH, and rainfall. The dataset is divided into training and testing sets, standardized using StandardScaler, and used to train the machine learning model.

Pest Risk Prediction Module

```
pipeline = Pipeline([
    ("preprocessor", ColumnTransformer([
        ("num", StandardScaler(), numeric_features),
        ("cat", OneHotEncoder(), categorical_features),
    ])),
    ("model", GaussianNB()),
])

pipeline.fit(X_train, y_train)
```

Listing 4.5 Code Snippet of Pest Risk Prediction

The pest risk prediction module is implemented using a Gaussian Naïve Bayes classifier. Numerical features are standardized, while categorical features are encoded using One-Hot Encoding before model training.

4.3 Testing

Software testing is a critical, continuous process used to evaluate a software product to determine if it meets specified requirements, functions correctly, and is ready for deployment. Its primary goal is to identify defects (bugs), assess the quality of the product, and ensure the final software provides the expected user experience. Basically the system is tested on two phases : Unit Testing (separately each components are tested) and System testing (components are integrated and tested).

4.3.1 Unit Testing

Unit tests targeted the riskiest functions: password hashing, JWT validation, permission resolution, and machine learning predictions.

Table 4.1 Unit Test Cases and Results

Test Case	Input	Expected Result	Actual Result
Register a farmer with valid details	name, email, password	201 Created; JWT returned; password stored as hash	Pass
Register with an existing email	duplicate email	400 Bad Request; duplicate email message	Pass
Hash the password before save	new Farmer document	password field contains a bcrypt hash	Pass
Compare correct password	correct candidate password	comparePassword returns true	Pass
Compare wrong password	incorrect candidate password	comparePassword returns false	Pass
Generate a valid JWT	farmer id	token verifies with JWT_SECRET and contains the id	Pass
Verify an expired token	token signed with short expiry	401 Invalid or expired token	Pass
Resolve role permissions	Farmer role document	Farmer permission set returned	Pass
Deny missing permission	request without products:manage_all	403 Insufficient permissions	Pass
Merge permission override	Farmer role plus an extra permission	merged set contains the override	Pass

4.3.2 System Testing

System testing executed complete user journeys in the browser, exercising every module together.

Table 4.2 System Test Cases and Results

Test Case	Steps	Expected Result	Actual Result
User registration and login	Register a new farmer, then log in	Account created; dashboard opens after login	Pass
Crop record submission	Fill crop form and submit	Record saved and shown in previous records	Pass
Pest risk prediction	Submit six pest features	Risk level with probability displayed	Pass
Crop recommendation	Submit seven soil/climate parameters	Recommended crop, tips, similar conditions shown	Pass
Disease recognition	Upload a diseased leaf image	Disease, cause, cure, and medicines shown	Pass
Shop and product detail	Browse shop and open a product	Approved products listed; detail page opens	Pass
Add to cart and checkout (COD)	Add product, fill address, pay on delivery	Order placed; stock decremented; order in history	Pass
Checkout with eSewa	Choose eSewa and pay on the gateway	Redirected to gateway; order verified and marked paid	Pass
Admin product approval	Approve a pending product as admin	Product becomes visible in the public shop	Pass
Admin farmer verification	Verify a farmer as admin	Verification status updated	Pass
Role and permission update	Add a permission to a role as admin	Role permissions updated and enforced	Pass

4.4 Result Analysis

The experimental results demonstrate that the machine learning models integrated into the Smart Agro System effectively support intelligent decision-making for modern agriculture. The system combines crop recommendation, pest risk prediction, and plant disease recognition to provide farmers with comprehensive agricultural assistance.

The crop recommendation model achieved excellent performance, with the Random Forest classifier obtaining a test accuracy of 99.55% on the held-out test dataset. The model consistently predicted suitable crops across twenty-two crop classes, with weighted precision, recall, and F1-score close to 1.00. Although the Gaussian Naïve Bayes model achieved the same accuracy, the Random Forest classifier was selected as the production model because of its robustness, ability to capture complex nonlinear relationships, and stronger generalization capability. The only notable misclassification occurred in the blackgram class, which achieved a recall of 0.95, while the remaining crop classes were classified with near-perfect accuracy. These results confirm that the crop recommendation module is reliable for supporting crop selection based on soil nutrients and environmental conditions.

The pest risk prediction model was implemented using a Gaussian Naïve Bayes pipeline. Due to the limited dataset containing only 17 samples, the model was evaluated using 3-fold cross-validation instead of relying solely on a hold-out test. The model achieved a mean cross-validation accuracy of 75.56%, with a standard deviation of 17.50%, indicating performance variation caused by the small dataset. A separate 80/20 hold-out evaluation resulted in an accuracy of 50.00%, but this result is not considered representative because it was based on only four test samples. The findings indicate that the pest prediction module can identify general pest risk patterns; however, its performance is constrained by the limited amount of training data. Expanding the dataset with more pest observations from different crops, seasons, and geographic regions is expected to significantly improve its prediction accuracy and reliability.

The disease recognition module was developed using a convolutional neural network, capable of classifying plant leaf images into 39 disease categories. The model processes 160×160 pixel images and returns the top three predicted disease classes with their corresponding confidence scores. To improve prediction reliability, a confidence threshold of 0.5 was applied, with images below this threshold classified as "Unrecognized." During transfer learning, the model converged successfully on the training and validation datasets, demonstrating its ability to learn disease characteristics effectively. Although an independently labeled test dataset was not available for quantitative evaluation, qualitative testing showed that the model consistently identified common plant diseases and provided appropriate treatment recommendations. The confidence threshold further reduced the likelihood of incorrect classifications by filtering uncertain predictions.

Overall, the experimental results demonstrate that the Smart Agro System successfully integrates multiple machine learning techniques to address key agricultural challenges. The crop recommendation module provides highly accurate crop suggestions, the pest risk prediction module offers preliminary pest risk assessment, and the disease recognition module enables early detection of plant diseases with corresponding treatment recommendations. Together, these intelligent services improve farmers' decision-making, support sustainable agricultural practices, and demonstrate the potential of artificial intelligence to enhance productivity and efficiency in modern farming.

Chapter-5. Conclusion and Expected Outcome

5.1 Conclusion

The Smart Agro System was able to be designed, developed, and tested successfully to offer a modern agriculture solution to the farmers through an intelligent, automated, and easy to use system. By integrating, real-time monitoring manual automation, pest alert detection and record management, the system improves the efficiency of farming, and facilitates the use of data to make decisions.

System testing helped to validate that all the modules such as authentication, pest detection, manual pump control, and record keeping work as it is expected. The functional and the non-functional requirements were verified and the application is reliable, usable as well as scalable.

5.2 Outcome

The Smart Agro System was successfully developed as an intelligent, web-based platform that integrates modern technologies to support farmers in managing agricultural activities more efficiently. The system provides essential features such as secure user authentication, crop recommendation based on soil characteristics, an online marketplace for buying and selling agricultural products, disease recognition through image analysis, and appropriate treatment recommendations for detected plant diseases.

The implemented solution simplifies farm management by allowing farmers to maintain digital records of their farming activities, monitor crop health, and make informed decisions using data-driven insights. The disease recognition module enables early detection of plant diseases, helping farmers take timely corrective actions to reduce crop losses and improve productivity.

The integrated marketplace enhances communication and trade between farmers, buyers, and sellers, creating better business opportunities while reducing dependence on traditional marketing channels. The crop recommendation feature assists farmers in selecting suitable crops according to soil conditions, contributing to improved resource utilization and higher agricultural yields.

Overall, the Smart Agro System achieves its primary objective of providing a reliable, user-friendly, and scalable digital platform for modern agriculture.

5.3 Future Recommendation

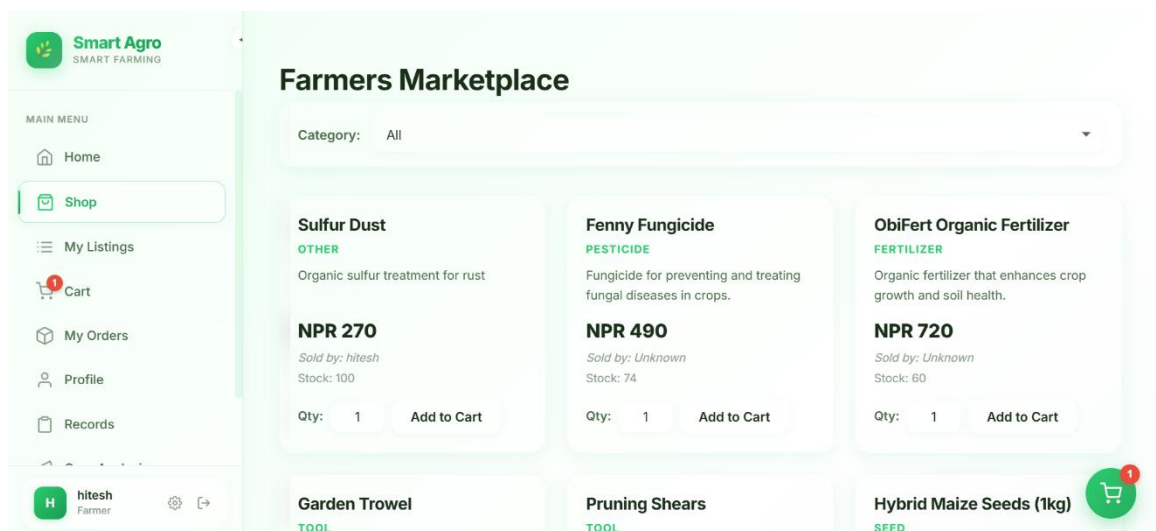
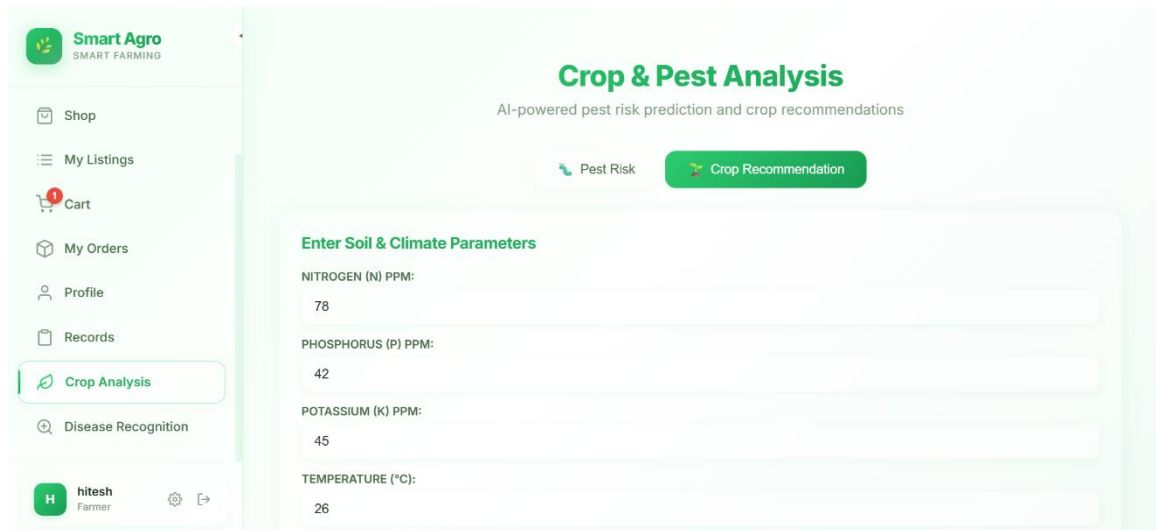
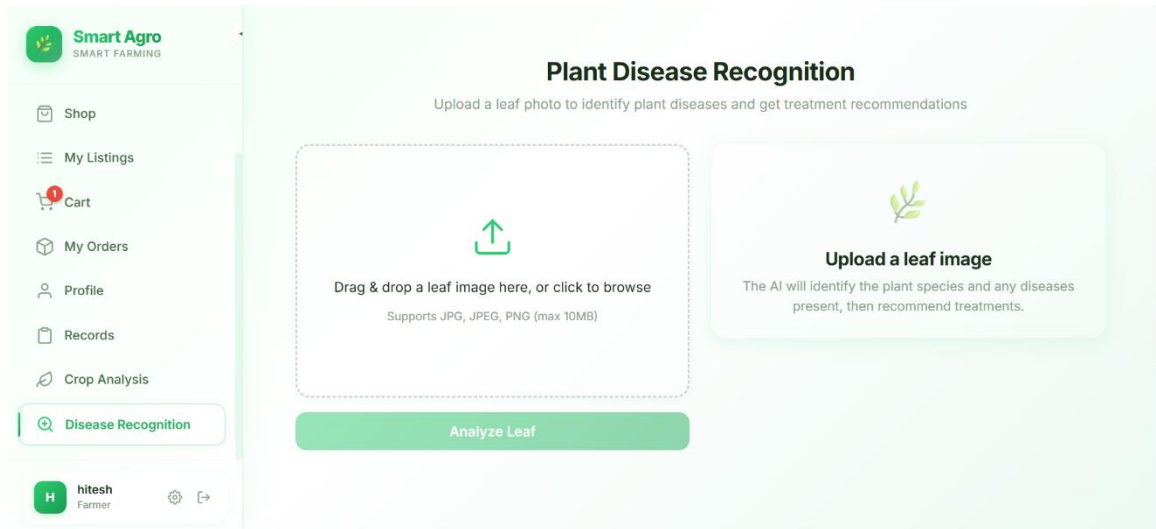
Even though Smart Agro System already provides useful features, there are several opportunities to make the system even better in the future:

- **Weather Forecasting Integration:** Integrating weather forecasting services will provide farmers with timely weather information, such as rainfall, temperature, and storms. This will help farmers make better decisions about irrigation, planting, harvesting, and other farming activities.
- **Farmer Community Feature:** A community platform can allow farmers to share their experiences, farming knowledge, successful practices, and solutions to common problems. This can encourage collaboration and knowledge sharing among farmers.
- **Mobile Application:** Developing Android and iOS mobile applications will make the Smart Agro System easier to access from the field. Farmers can conveniently receive recommendations, monitor farm information, and use system features directly through their smartphones.

References

- [1] N. W. M. & W. N. Zhang, "Precision agriculture: A comprehensive approach," *Computers and Electronics in Agriculture*, 2021. [https://doi.org/10.1016/S0168-1699\(02\)00096-0](https://doi.org/10.1016/S0168-1699(02)00096-0)
- [2] R. & S. P. Patel, "Machine learning approaches for agricultural recommendation," *Journal of Agricultural Informatics*, 2020. <https://doi.org/10.1038/s41598-025-88676>
- [3] A. S. S. & T. S. K. Kumar, "Comparative analysis of machine learning algorithms," *International Journal of Agricultural and Environmental*, 2019. <https://doi.org/10.1016/j.rineng.2025.105505>
- [4] S. & V. H. K. Gupta, "Intelligent pest detection in crops using machine learning," *Precision Agriculture*, 2022. <https://doi.org/10.1016/j.procs.2025.03.346>
- [5] X. & C. Y. Li, "Bayesian methods for agricultural decision support systems," *Agricultural Systems*. <https://doi.org/10.1016/j.compag.2020.105709>
- [6] A. & G. T. Brown, "Adoption barriers for Smart Agro technologies in developing," *Technology in Society*, 2021. <https://doi.org/10.1016/j.heliyon.2024.e40836>
- [7] P. & M. C. Anderson, "Economic impacts of precision agriculture technologies," *Agricultural Economics*, 2020. <https://doi.org/10.1016/j.compag.2018.01.009>
- [8] E. & W. B. Thompson, "Environmental benefits of Smart Agro systems," *Journal of Cleaner Production*, 2021. <https://doi.org/10.1016/j.future.2021.08.006>
- [9] H. L. R. & Z. W. Chen, "Future trends in agricultural robotics and automation," *Agricultural Systems*, 2022. <https://doi.org/10.1109/ICCCI54379.2022.9741001>

Appendices



 **Smart Agro**
SMART FARMING

MAIN MENU

Home

Shop

My Listings

Cart

My Orders

Profile

Records

hitesh

Farmer

 **Smart Farming**

Sensor Graphs

Air Humidity Graph

Air Humidity Values

1.0

0.8

0.6

0.4

0.2

0

Air Humidity Values

Time

Soil Moisture Graph

Soil Moisture Values

1.0

0.8

0.6

0.4

0.2

0

Soil Moisture Values

Time

Light Intensity Graph

Light Intensity Values

1.0

0.8

0.6

0.4

0.2

0

Light Intensity Values

Time

 **Admin Panel**
SMART FARMING

Dashboard

Users

Farmers

Products

Medicines

Orders

A

Admin

Admin

Smart Farming

Admin Dashboard

Overview

Crop Records

Roles

Total Users

3

Total Crop Records

6

Roles

2

Recent Users

hitesh

hiteshdangol@gmail.com

Farmer

farmer@farmer.com

Recent Crop Records

rice

20 kg

kg

Smart Agro

Welcome back, farmer

admin@admin.com

Login

Don't have an account? [Register here](#)

Source Code

```
exports.registerFarmer = async (req, res) => {
  try {
    const { name, email, password, role } = req.body;
    if (!name || !email || !password) {
      return res.status(400).json({ success: false, message: 'Please provide name, email, and password' });
    }
    const existingFarmer = await Farmer.findOne({ email: email.toLowerCase() });
    if (existingFarmer) {
      return res.status(400).json({ success: false, message: 'Farmer with this email already exists' });
    }
    const defaultRole = await Role.findOne({ isDefault: true });
    const farmer = await Farmer.create({
      name: name.trim(),
      email: email.toLowerCase().trim(),
      password,
      role: role || (defaultRole ? defaultRole.name : 'Farmer')
    });
    const token = generateToken(farmer._id);
    res.status(201).json({ success: true, message: 'Farmer registered successfully', token, farmer });
  } catch (error) {
    if (error.name === 'ValidationError') {
      const messages = Object.values(error.errors).map(err => err.message);
      return res.status(400).json({ success: false, message: 'Validation Error', errors: messages });
    }
    if (error.code === 11000) {
      return res.status(400).json({ success: false, message: 'Email already exists' });
    }
    res.status(500).json({ success: false, message: 'Server error during registration' });
  }
};
```

```
exports.loginFarmer = async (req, res) => {
  try {
    const { email, password } = req.body;
    if (!email || !password) {
      return res.status(400).json({ success: false, message: 'Please provide email and password' });
    }
    const farmer = await Farmer.findOne({ email: email.toLowerCase().trim() }).select('+password');
    if (!farmer) {
      return res.status(401).json({ success: false, message: 'Invalid email or password' });
    }
    const isPasswordValid = await farmer.comparePassword(password);
    if (!isPasswordValid) {
      return res.status(401).json({ success: false, message: 'Invalid email or password' });
    }
    const token = generateToken(farmer._id);
    farmer.password = undefined;
    res.status(200).json({ success: true, message: 'Login successful', token, farmer });
  } catch (error) {
    res.status(500).json({ success: false, message: 'Server error during login' });
  }
};
```

```
const generateToken = (id) =>
  jwt.sign({ id }, process.env.JWT_SECRET, { expiresIn: process.env.JWT_EXPIRE || '7d' });
Listing E.4 — backend/utls/authMiddleware.js + permissionMiddleware.js
// authMiddleware.js
const authMiddleware = async (req, res, next) => {
  const token = req.header('Authorization')?.split(' ')[1];
  if (!token) {
    return res.status(401).json({ message: 'No token, authorization denied.' });
  }
  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    const farmer = await Farmer.findById(decoded.id).select('-password');
    if (!farmer) {
      return res.status(401).json({ message: 'Authorization denied: user not found.' });
    }
    req.user = farmer;
    next();
  } catch (err) {
    res.status(401).json({ message: 'Invalid or expired token.' });
  }
};
```

```

    }
  };

  // permissionMiddleware.js
  const requirePermission = (...requiredPerms) => {
    return async (req, res, next) => {
      try {
        const role = await Role.findOne({ name: req.user.role });
        const rolePerms = role ? role.permissions : [];
        const merged = new Set([...rolePerms, ...(req.user.permissionsOverride || [])]);
        req.permissions = [...merged];
        const hasAll = requiredPerms.every(p => merged.has(p));
        if (!hasAll) {
          return res.status(403).json({ success: false, message: 'Insufficient permissions' });
        }
        next();
      } catch (error) {
        res.status(500).json({ success: false, message: error.message });
      }
    };
  };
};

```

Listing E.5 — backend/controllers/orderController.js (eSewa flow)

```

exports.initiateEsewaPayment = async (req, res) => {
  try {
    const { orderId } = req.body;
    const order = await Order.findById(orderId);
    if (!order) return res.status(404).json({ success: false, message: 'Order not found' });
    if (order.buyer.toString() !== req.user.id) return res.status(403).json({ success: false, message: 'Unauthorized' });

    const totalAmount = Math.round(order.totalAmount).toString();
    const transactionUuid = `${orderId}_${Date.now()}`;
    const productCode = process.env.ESEWA_MERCHANT_ID || 'EPAYTEST';
    const secretKey = process.env.ESEWA_SECRET_KEY;

    const message = `total_amount=${totalAmount},transaction_uuid=${transactionUuid},product_code=${productCode}`;
    const hmac = crypto.createHmac('sha256', secretKey);
    hmac.update(message);
    const signature = hmac.digest('base64');

    const formData = {
      amount: totalAmount, tax_amount: '0', total_amount: totalAmount,
      transaction_uuid: transactionUuid, product_code: productCode,
      product_service_charge: '0', product_delivery_charge: '0',
      success_url: process.env.ESEWA_SUCCESS_URL,
      failure_url: process.env.ESEWA_FAILURE_URL,
      signed_field_names: 'total_amount,transaction_uuid,product_code',
      signature,
    };
    res.json({ success: true, formData, esewaUrl: 'https://rc-epay.esewa.com.np/api/epay/main/v2/form' });
  } catch (error) {
    res.status(500).json({ success: false, message: error.message });
  }
};

exports.verifyEsewaPayment = async (req, res) => {
  try {
    const { orderId, transaction_code, total_amount, transaction_uuid } = req.body;
    const productCode = process.env.ESEWA_MERCHANT_ID || 'EPAYTEST';
    const url = `https://rc-epay.esewa.com.np/api/epay/transaction/status/?product_code=${productCode}&total_amount=${total_amount}&transaction_uuid=${transaction_uuid}`;
    const response = await new Promise((resolve, reject) => {
      https.get(url, (resp) => {
        let data = '';
        resp.on('data', chunk => data += chunk);
        resp.on('end', () => { try { resolve(JSON.parse(data)); } catch { resolve(data); } });
      }).on('error', reject);
    });
    if (response.status === 'COMPLETE' || response?.state === 'completed') {
      const order = await Order.findById(orderId);
      order.paymentStatus = 'paid';
      order.transactionId = transaction_code || response.transaction_code;
      await order.save();
      for (const item of order.items) {
        await Product.findByIdAndUpdate(item.product, { $inc: { stock: -item.quantity } });
      }
    }
  }
};

```

```

    }
    return res.json({ success: true, order, message: 'Payment verified successfully' });
  }
  res.status(400).json({ success: false, message: 'Payment verification failed', details: response });
} catch (error) {
  res.status(500).json({ success: false, message: error.message });
}
};

```

Listing E.6 — pythonmodel/api_nb.py (pest risk service)

```
model = joblib.load(os.path.join(os.path.dirname(__file__), "pest_nb_pipeline.joblib"))
```

```

class PestInput(BaseModel):
    temp_avg: float
    humidity: float
    rainfall_mm: float
    crop_type: str
    growth_stage: str
    prev_pest_incidence: float

```

```

@app.post("/predict")
def predict_pest_risk(data: PestInput):
    if model is None:
        return {"error": "Model not loaded"}
    df = pd.DataFrame([data.dict()])
    prediction = model.predict(df)[0]
    return {"risk_level": prediction}

```